

PROGRAMACIÓN MULTIMEDIA Y DISPOSITIVOS MÓVILES

UD 1 Introducción a la programación móvil

ÍNDICE DE CONTENIDO

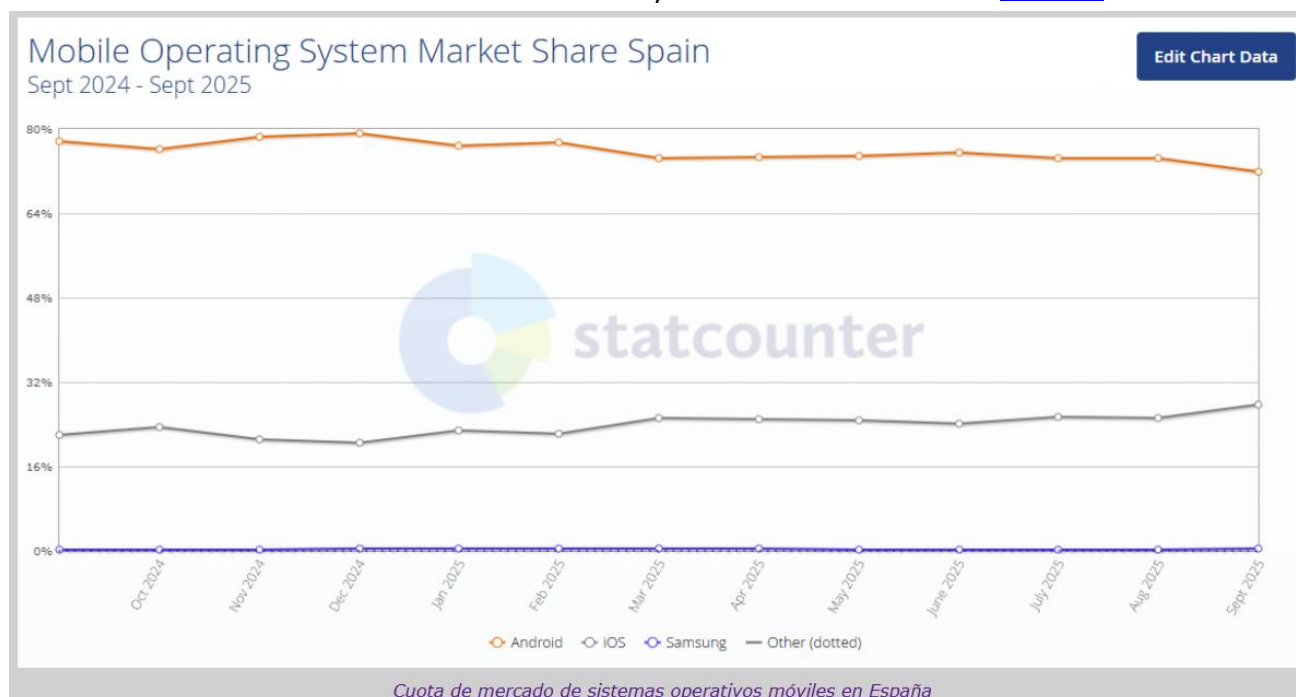
1.	Introducción	3
2.	Tipos de Aplicaciones	4
2.1	Ventajas del desarrollo de aplicaciones nativas	4
2.2	Desventajas de las aplicaciones nativas	5
2.3	Ventajas del desarrollo multiplataforma	5
2.3	Desventajas del desarrollo multiplataforma	6
3.	Características de Android	6
4.	Arquitectura de Android	7
5.	Fragmentación	8
6.	Android Studio	11
6.1	Introducción	11
6.2	INSTALACIÓN Y CONFIGURACIÓN DEL ENTORNO	12
6.3	INSTALACIÓN DE ANDROID STUDIO (AS)	13
6.4	Hola Mundo. Mi primera aplicación	21
7.	Emulador de Android (AVD - Android Virtual Device)	27
8.	Pruebas en Dispositivos Reales	32
8.1	Instalación de driver USB, Modo Desarrollador y Depuración USB	32
8.2	Depuración inalámbrica	35
9.	Emuladores de Terceros	36
9.1	BlueStacks 10	36
9.2	Genymotion	42
10.	Otros Sistemas.	42
11.	Bibliografía	42

INTRODUCCIÓN A LA PROGRAMACIÓN MÓVIL

1. INTRODUCCIÓN

En 25 años, el mundo ha cambiado como nunca, y ahora el mundo es móvil. Quienes hemos tenido móviles desde el principio hemos pasado de tener un teléfono analógico ETACS como MovilLine a los modernos dispositivos digitales 5G. Hoy cualquiera puede hablar y navegar desde cualquier lugar. Por eso a los menores de 20 se les llama nativos digitales. No sorprende que los estudios muestren que en España el móvil superó al PC de sobremesa como dispositivo principal de acceso a Internet ya en 2017.

Dentro del mundo móvil, principalmente tenemos las plataformas iOS de Apple, BlackBerry y Android (tras el fracaso de Windows Mobile Phone de Microsoft y Symbian de Nokia). Por desgracia, las dos primeras son propietarias, lo que ha llevado a Android a dominar el mercado, como se ve en este [enlace](#).



La comparación de características entre Android e iOS puede resumirse así...

Android	iOS
Altamente personalizable, pero requiere tiempo adquirir conocimientos para personalizar el teléfono.	Es muy especializado y la plataforma lo controla todo. Funciona bien para quien está conforme con la configuración por defecto.
Debido a la fragmentación, las actualizaciones dependen del modelo y fabricante de cada dispositivo.	Los usuarios tienen acceso siempre a la última versión de iOS, gracias a actualizaciones gratuitas y automáticas.
Disponible en teléfonos de varios fabricantes: HTC, Google, Samsung, Motorola, LG, etc. Gran variedad de modelos.	Solo disponible en dispositivos Apple: iPod touch, iPad, iPhone y Apple TV.
Cualquiera puede crear y distribuir aplicaciones Android. Los usuarios pueden descargar e instalar apps fuera de la tienda oficial.	Es más caro para desarrolladores publicar apps para iOS. La App Store es prácticamente el único lugar desde el que descargar apps.

2. TIPOS DE APLICACIONES

El mercado de telefonía móvil es extremadamente diverso. Cada mes se lanzan decenas de nuevos smartphones y, como resultado, los desarrolladores y emprendedores se enfrentan a retos por el creciente número de aplicaciones móviles. No existe una solución mágica para elegir la plataforma a utilizar.

Tanto las aplicaciones nativas como las multiplataforma tienen sus ventajas y desventajas. Como responsable de un negocio, conviene entender las diferencias entre ellas..

2.1 Ventajas del desarrollo de aplicaciones nativas

- Acceso al potencial completo del dispositivo
La esencia de las apps nativas es que se escriben para una plataforma específica (normalmente, Android o iOS). Esto permite ofrecer mucha mejor integración entre las funcionalidades de la app y el hardware del móvil: Bluetooth, NFC, GPS, cámara, micrófono, acelerómetro, giroscopio, etc.

- **Mejor rendimiento en pantalla**
Usar un lenguaje nativo te da más libertad en cuanto a optimización de pantalla. Con la cantidad de dispositivos y sus distintos tamaños y relaciones de aspecto, es crucial asegurar que tu app se muestre correctamente en todas partes.
- **Mejor rendimiento**
Junto con el control y la flexibilidad, la mejor integración con el hardware aporta otra ventaja importante. Las apps nativas son muy rápidas y rinden mejor que otras, como las web. Las apps nativas aprovechan al máximo la potencia de proceso, minimizando el tiempo de respuesta.
- **No requieren conexión a Internet.**
Como las apps nativas se instalan directamente en el dispositivo, pueden acceder a datos sin conexión. En cambio, una app web u otras plataformas requieren conexión constante, dependiente de la red.

Combinado con el rendimiento del punto anterior, la capacidad de funcionar offline hace que las apps nativas sean la elección perfecta para juegos móviles, por ejemplo.

2.2 Desventajas de las aplicaciones nativas

Si hay una desventaja en el desarrollo nativo, es la incompatibilidad con otros sistemas operativos móviles. Las apps nativas de iOS se desarrollan con Swift u Objective-C y solo se pueden instalar en iPhone o iPad. Del mismo modo, las apps Android no son compatibles con iOS porque se desarrollan en Java o Kotlin.

Otro aspecto a mencionar es la compatibilidad de versiones. Por ejemplo, dada la gran variedad de versiones de Android, dar soporte completo a todos los usuarios puede ser un reto

2.3 Ventajas del desarrollo multiplataforma

- **Código reutilizable**
En lugar de desarrollar distintas apps, los desarrolladores pueden usar el mismo código para todas las plataformas. Esto significa que, a diferencia de una app nativa, con desarrollo multiplataforma implementas una funcionalidad en una app para un SO y funcionará en los demás.

Esta reutilización también ofrece ventaja en el despliegue de la app.

- **Rentabilidad**

Implementar tu idea como app multiplataforma no requiere grandes inversiones. No necesitas un equipo dedicado por cada sistema operativo. Gracias a una curva de aprendizaje corta y un conjunto unificado de tecnologías, un único equipo puede dar soporte a todas las plataformas.

- La misma UI y UX en todas las plataformas

El rendimiento es importante, pero también lo es cómo se ve y fluye la app. Puede ser difícil lograr la misma interfaz y experiencia de usuario porque intervienen dos sistemas independientes.

Usando una sola base de código, las apps multiplataforma mantienen la misma apariencia y experiencia en todos los dispositivos.

2.3 Desventajas del desarrollo multiplataforma

- No son tan flexibles como las nativas.
- El rendimiento no es su punto fuerte.
- El proceso de publicarlas en tiendas puede ser más complicado.

3. CARACTERÍSTICAS DE ANDROID

Aunque hay varias plataformas móviles, el sistema operativo Android ofrece un conjunto de características que lo diferencian del resto:

- Plataforma abierta. Es una plataforma de desarrollo gratuita, basada en Linux y de código abierto. Entre sus ventajas está poder usar y adaptar el sistema sin pagar royalties.
- Portabilidad garantizada. Gracias al concepto de máquina virtual, las apps finales se desarrollan en Java, garantizando que se ejecuten en una gran variedad de dispositivos, presentes y futuros.
- Arquitectura basada en componentes inspirados en Internet. Por ejemplo, el diseño de la interfaz de usuario se hace en XML, permitiendo que la misma app se ejecute en un móvil con pantalla pequeña o en un Netbook.
- Filosofía de dispositivo siempre conectado a Internet.
- Muchos servicios integrados: localización por GPS, bases de datos SQL, reconocimiento y síntesis de voz, navegador, mapas, etc.
- Nivel de seguridad aceptable. Los programas están aislados gracias al concepto de sandbox proporcionado por la máquina virtual.
- Optimizado para bajo consumo y poca memoria. Android usaba la máquina virtual Dalvik, implementación de Google de la JVM optimizada para móviles.
- Gráficos y sonido de alta calidad. Gráficos vectoriales fluidos, animaciones inspiradas en Flash, 3D basado en OpenGL y códecs estándar de audio y

vídeo.

Android es un sistema operativo orientado a dispositivos móviles. La empresa, como muchas tecnológicas, fue fundada en Palo Alto (California) en 2003. Debido a su rápido crecimiento y potencial, llamó la atención de Google, que la compró en 2005 por 50 millones de dólares.

4. ARQUITECTURA DE ANDROID

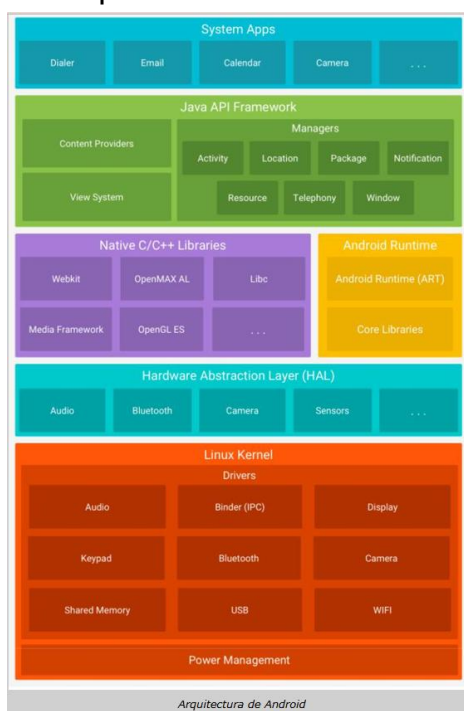
Android implementa un sistema operativo basado en Linux, y su máquina virtual (Dalvik) compila archivos Java en aplicaciones .apk. El formato DEX es independiente de la arquitectura del procesador, así que para ejecutarse debe traducirse a código nativo para el procesador del dispositivo. Naturalmente, esto es más lento que la ejecución directa.

*Actualmente se usa AAB (Android App Bundle).

Para resolverlo, Google introdujo una nueva máquina virtual llamada ART a partir de KitKat (4.4).

La diferencia fundamental entre Dalvik y ART es que Dalvik usa compilación just-in-time (JIT), mientras que ART usa compilación ahead-of-time (AOT). Dalvik ejecuta una VM que interpreta código mientras corre la app, mientras que ART realiza precompilación durante la instalación de la app.

Entre otras diferencias, ART ofrece mejoras en el recolector de basura, así como en desarrollo y depuración de aplicaciones.



Como se ve, Android usa numerosas bibliotecas internas, como SGL, OpenGL, OpenCORE, WebKit, SSL, FreeType, SQLite y libc.

Hoy Android está presente en todo tipo de dispositivos, principalmente smartphones, tablets, relojes (Wear OS), Android TV, Android Auto, etc. Incluso con el proyecto Android-x86 es posible usar Android de código abierto instalado en un PC, como la versión adaptada de Android 9 Pie.

Uno de los principales retos para desarrolladores es la enorme cantidad de dispositivos, formatos de pantalla y orientaciones de uso, además de las distintas densidades de pantalla. Al diseñar una app, debemos decidir en qué tipos de dispositivos funcionará con fluidez y cuántos diseños de pantalla distintos necesitaremos.

Otro reto importante es la fragmentación: coexisten demasiadas versiones en el mercado, cada una con sus peculiaridades, y se liberan tan rápido que cuesta seguir el ritmo. Por tanto, al diseñar una app hay que elegir desde qué versión funcionará. Existen librerías de soporte para usar características modernas en dispositivos antiguos.

Una peculiaridad poco conocida por no desarrolladores es que, en Android, la versión del SO no siempre coincide con el nivel de API que debemos usar para crear una app para esa versión. Veremos esta relación en seguida

5. FRAGMENTACIÓN

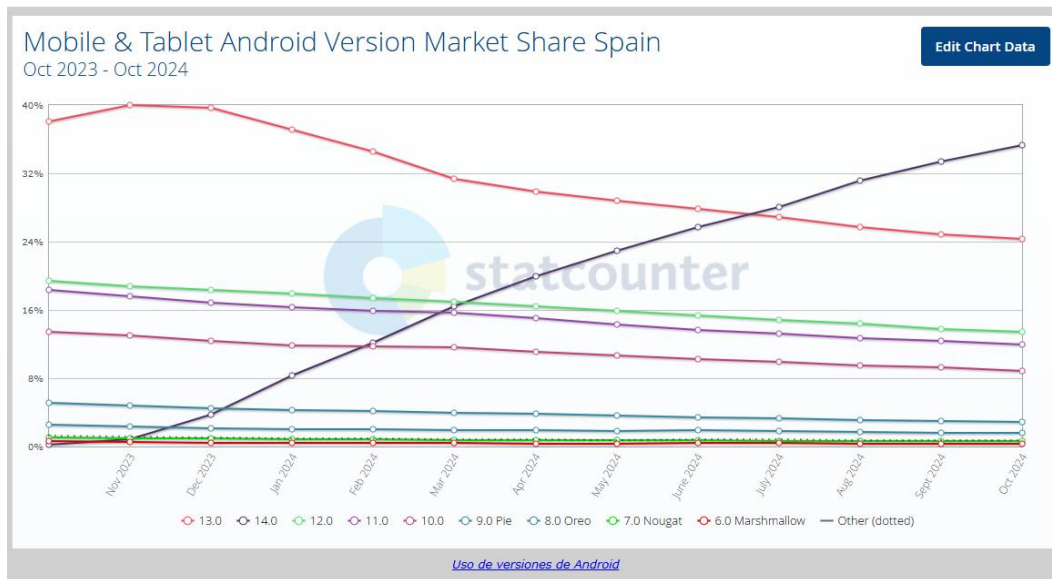
La fragmentación de Android se refiere a las distintas versiones del mismo sistema operativo. Como en otros sistemas, Android evoluciona añadiendo mejoras de seguridad y nuevas funciones, y para diferenciarlas se crean versiones.

En Android debemos distinguir entre el número de versión y el nivel de API. Al desarrollar aplicaciones, debemos considerar el nivel de API seleccionado para ver qué versiones o dispositivos podrán ejecutarla. Aun así, existen librerías para mantener compatibilidad hacia atrás con versiones previas.

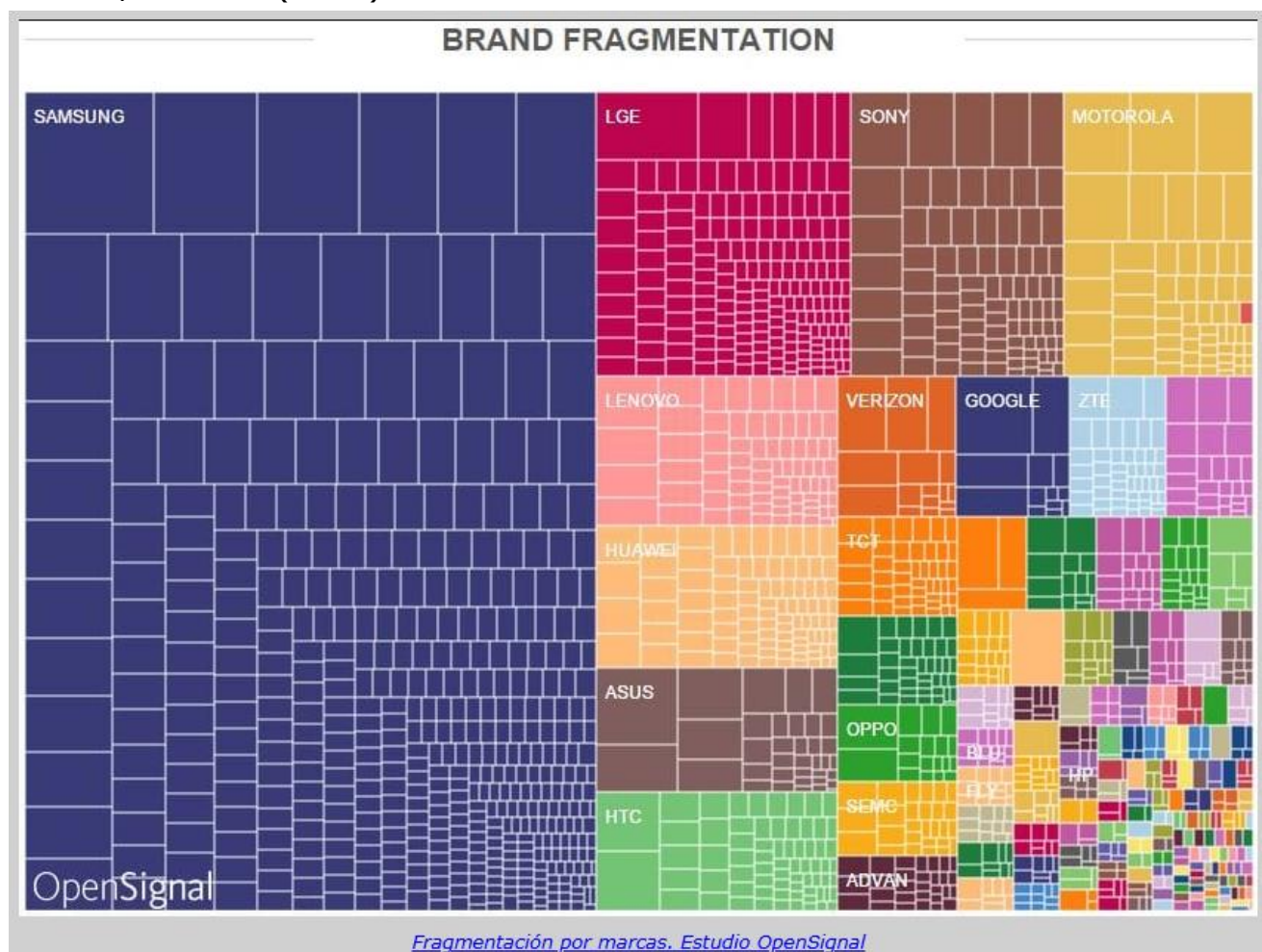
Nombre	Nombre interno	Número(s) de versión	Nivel de API	Fecha de lanzamiento	Último parche de seguridad	Última versión de Servicios de Google Play
Android 1.0	—	1.0	1	September 23, 2008	—	—
Android 1.1	Petit Four	1.1	2	February 9, 2009		
Android Cupcake	Cupcake	1.5	3	April 27, 2009		
Android Donut	Donut	1.6	4	September 15, 2009		
Android Eclair	Eclair	2.0	5	October 27, 2009		
		2.0.1	6	December 3, 2009		
		2.1	7	January 11, 2010		
Android Froyo	Froyo	2.2 – 2.2.3	8	May 20, 2010		3.2.25 (October 2014)
Android Gingerbread	Gingerbread	2.3 – 2.3.2	9	December 6, 2010		10.0.84 (November 2016)
		2.3.3 – 2.3.7	10	February 9, 2011		
Android Honeycomb	Honeycomb	3.0	11	February 22, 2011		
		3.1	12	May 10, 2011		
		3.2 – 3.2.6	13	July 15, 2011		
Android Ice Cream Sandwich	Ice Cream Sandwich	4.0 – 4.0.2	14	October 18, 2011		14.8.49 (February 2019)
		4.0.3 – 4.0.4	15	December 16, 2011		
Android Jelly Bean	Jelly Bean	4.1 – 4.1.2	16	July 9, 2012		21.33.56 (September 2021)
		4.2 – 4.2.2	17	November 13, 2012		
		4.3 – 4.3.1	18	July 24, 2013		
Android KitKat	Key Lime Pie	4.4 – 4.4.4	19	October 31, 2013	October 2017	23.30.13 (August 2023)
		4.4W – 4.4W.2	20	June 25, 2014	?	
Android Lollipop	Lemon Meringue Pie	5.0 – 5.0.2	21	November 4, 2014	November 2017	24.28.35 (August 2024)
		5.1 – 5.1.1	22	March 2, 2015	March 2018	
Android Marshmallow	Macadamia Nut Cookie	6.0 – 6.0.1	23	September 29, 2015	August 2018	24.35.30 (September 2024)
Android Nougat	New York Cheesecake	7.0	24	August 22, 2016	August 2019	
		7.1 – 7.1.2	25	October 4, 2016	October 2019	
Android Oreo	Oatmeal Cookie	8.0	26	August 21, 2017	January 2021	
		8.1	27	December 5, 2017	October 2021	
Android Pie	Pistachio Ice Cream	9	28	August 6, 2018	January 2022	
Android 10	Quince Tart	10	29	September 3, 2019	February 2023	
Android 11	Red Velvet Cake	11	30	September 8, 2020	February 2024	
Android 12	Snow Cone	12	31	October 4, 2021	September 2024	
Android 12L	Snow Cone v2	12.1	32	March 7, 2022		
Android 13	Tiramisu	13	33	August 15, 2022		
Android 14	Upside Down Cake	14	34	October 4, 2023		
Android 15	Vanilla Ice Cream	15	35	October 15, 2024		
Android 16	Baklava	16 DP 1	36	November 19, 2024		
Legend: Old version, not maintained Old version, still maintained Latest version						

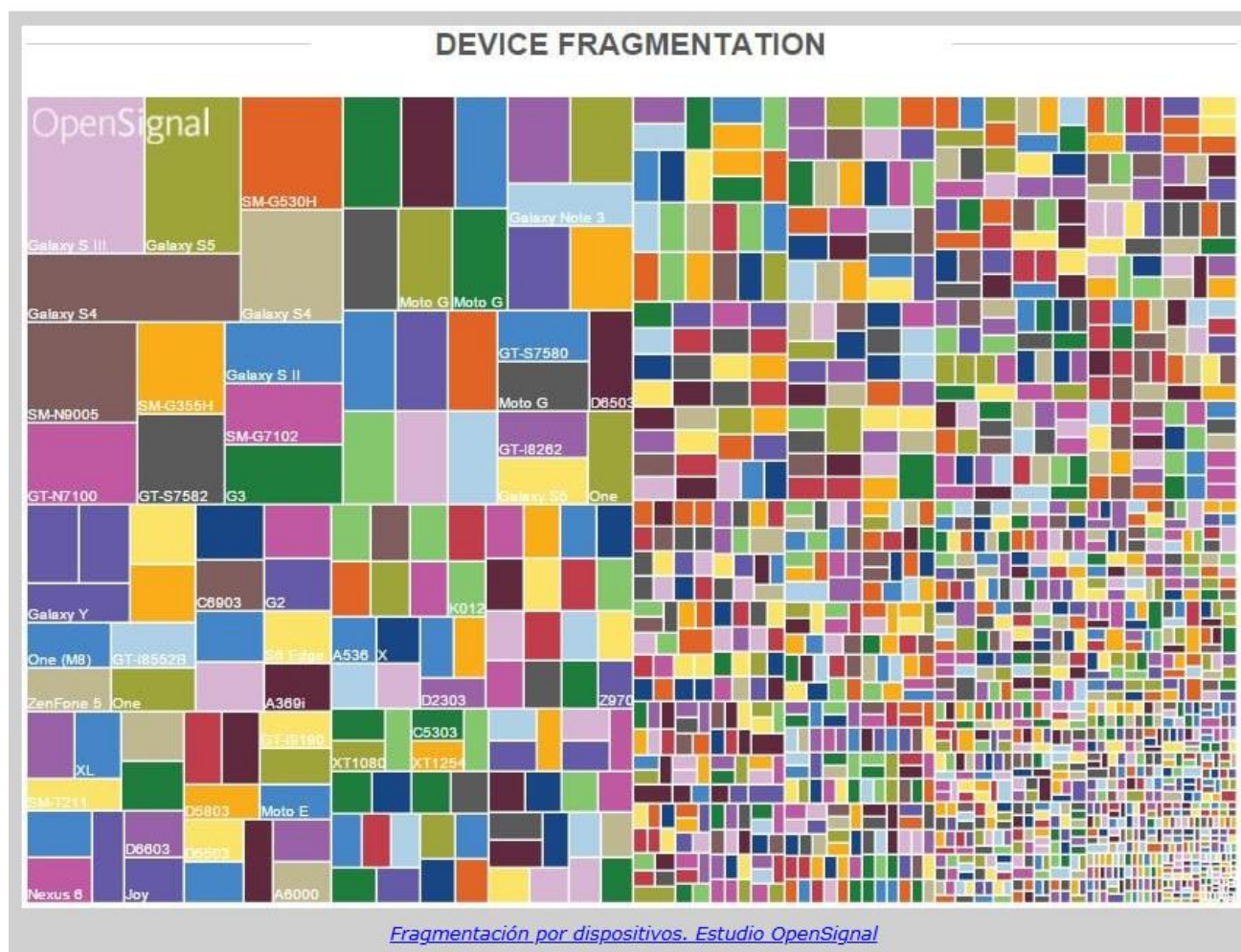
Legend: Old version, not maintained Old version, still maintained Latest version

Versões de Android / Nivel de API



Para concluir esta sección, cabe mencionar la fragmentación que existe entre marcas/modelos (2015).





6. ANDROID STUDIO

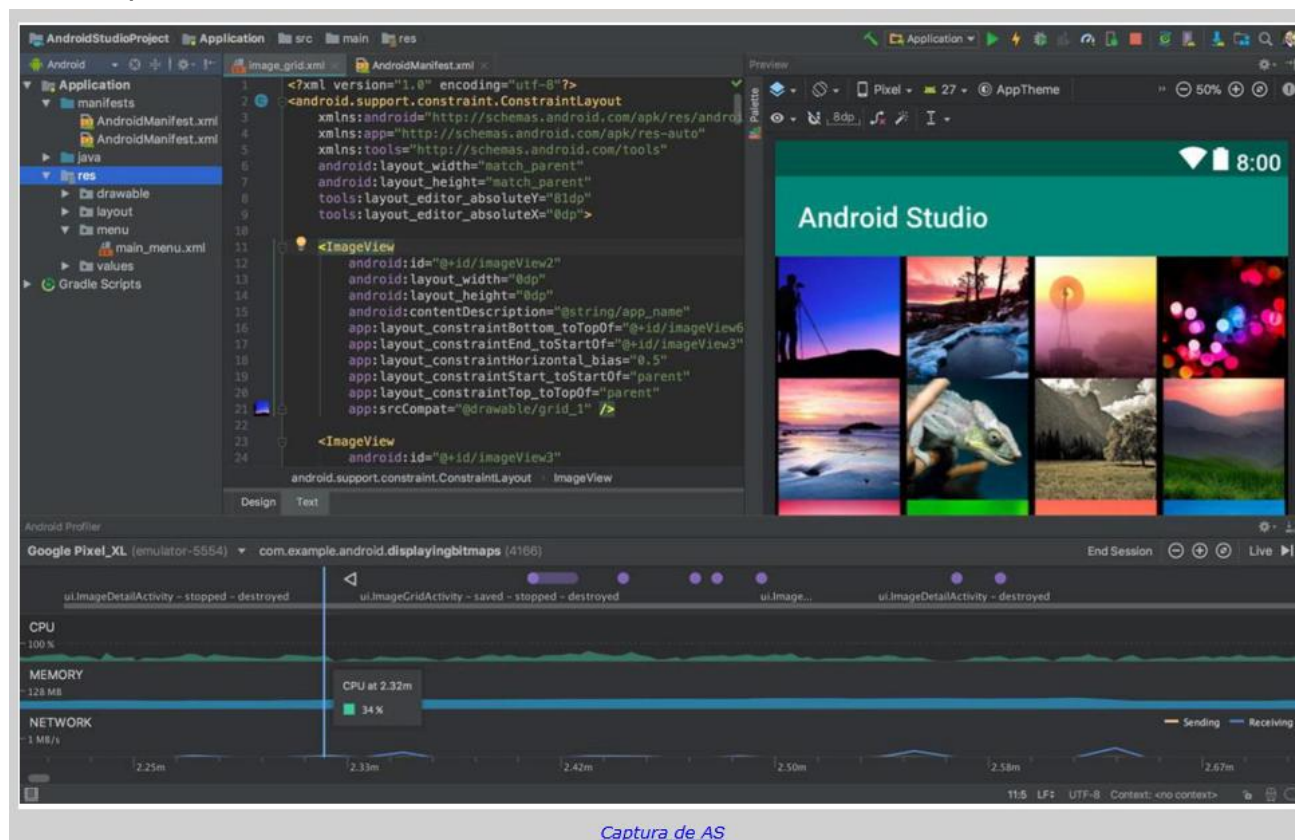
6.1 Introducción

Android Studio (AS) es el IDE oficial de Google para crear aplicaciones que se ejecutan en su sistema operativo. Sustituyó al antiguo entorno oficial, Eclipse (<https://www.eclipse.org/>), y permite desarrollar apps con Java o Kotlin (<https://kotlinlang.org/>).

A diferencia de AppInventor (<http://appinventor.mit.edu/>), que facilita generar apps mediante programación por bloques, con Android Studio hay que trabajar desde cero. Efectivamente, trabajar con AS es laborioso, pero a cambio tenemos un control claro y casi total de lo que construimos.

Las aplicaciones creadas con PhoneGap (<https://phonegap.com/>) o su “hermano” Apache Cordova (<https://cordova.apache.org/>) son híbridas (HTML / CSS / JS), mientras que las creadas con AS serán totalmente nativas, con acceso completo a los recursos de hardware del sistema.

La versión actual (Android Studio Narwhal 4 Feature Drop | 2025.1.4) se muestra abajo. Principalmente vemos la barra superior de menús, el árbol del proyecto a la izquierda, el código en la zona central con pestañas y el diseño de interfaz a la derecha. También se muestra el consumo de recursos; normalmente usaremos el área de notificaciones para Event Log, Logcat y Terminal. La ventana de paleta de componentes o sus atributos también es clave.



NOTA: A lo largo del curso, nos remitiremos intensamente a la documentación oficial de Android.

URLs: Introducción oficial a Android Studio y guías para desarrolladores.

<https://developer.android.com/studio/intro>

<https://developer.android.com/guide>

6.2 INSTALACIÓN Y CONFIGURACIÓN DEL ENTORNO

Para seguir el curso, necesitamos instalar el IDE Android Studio en nuestro sistema operativo preferido, que incluye el SDK de Android con APIs integradas y librerías Java, el driver USB para pruebas en dispositivo real y, opcionalmente, algún software de emulación externo.

Aunque no será necesario, se pueden instalar componentes para trabajar offline (plugin de Gradle y dependencias de Maven), junto con herramientas de línea de

comandos (según el sistema operativo). Hoy en día solemos tener conexión constante, así que no es imprescindible.

6.3 INSTALACIÓN DE ANDROID STUDIO (AS)

Hay versiones de AS para Windows, Linux, Mac y Chrome. Puedes usar la que prefieras según tu sistema.

Enlace de descarga:

<https://developer.android.com/studio?hl=es&gclid=aw.ds#downloads>

Plataforma	Paquete de Android Studio	Tamaño	Suma de comprobación SHA-256
Windows (64 bits)	android-studio-2024.2.1.11-windows.exe Recomendado	1.2 GB	0b4433794d816fc3062d7ec8507a203580d991f5e8ddd51cc71a78925493a16d
Windows (64 bits)	android-studio-2024.2.1.11-windows.zip Sin instalador .exe	1.2 GB	16360e5d95312fa1f0c50866a4a3e89e425aa372e1d1511cecc204d31f8069df
Mac (64 bits)	android-studio-2024.2.1.11-mac.dmg	1.3 GB	9cb628c6b914e320570f5c54bb0e2adaa21757390233dce251e1fc4c21355035
Mac (64 bits, ARM)	android-studio-2024.2.1.11-mac_arm.dmg	1.2 GB	e72b838782c6b021d2a1ce6ae75e2668dbb7f9d245413a3b9fb6f246798fce34
Linux (64 bits)	android-studio-2024.2.1.11-linux.tar.gz	1.3 GB	81f2fa5da72ab6f6afacca4a5327de5191e133ad02706e3ee440dc8658b8cdc3
ChromeOS	android-studio-2024.2.1.11-cros.deb	1.0 GB	f4d5e6a263d07b4e535a3c3777263449820bad2fa2b66734494b40cf986af91d

[Versiones de Android Studio](#)

Los requisitos de hardware actuales para cada versión son:

System requirements

Windows

- 64-bit Microsoft® Windows® 8/10
- x86_64 CPU architecture; 2nd generation Intel Core or newer, or AMD CPU with support for a [Windows Hypervisor](#)
- 8 GB RAM or more
- 8 GB of available disk space minimum (IDE + Android SDK + Android Emulator)
- 1280 x 800 minimum screen resolution

Mac

- MacOS® 10.14 (Mojave) or higher
- ARM-based chips, or 2nd generation Intel Core or newer with support for [Hypervisor.Framework](#)
- 8 GB RAM or more
- 8 GB of available disk space minimum (IDE + Android SDK + Android Emulator)
- 1280 x 800 minimum screen resolution

Linux

- Any 64-bit Linux distribution that supports Gnome, KDE, or Unity DE; GNU C Library (glibc) 2.31 or later.
- x86_64 CPU architecture; 2nd generation Intel Core or newer, or AMD processor with support for AMD Virtualization (AMD-V) and SSE3
- 8 GB RAM or more
- 8 GB of available disk space minimum (IDE + Android SDK + Android Emulator)
- 1280 x 800 minimum screen resolution

Chrome OS

For information on recommended devices and specifications, as well as Android Emulator support, visit [chromeos.dev](#).

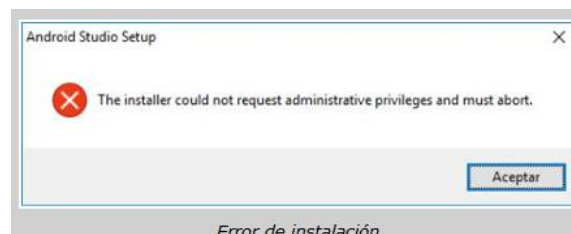
Requisitos de AS

En este curso usaremos la última versión disponible, Android Studio Ladybug | 2024.2.1 Patch 2 para Windows 64-bit. El motivo de usar Windows en las explicaciones es que suele dar menos problemas al reconocer el dispositivo real por USB para simulaciones reales usando el driver ADB u otros. No obstante, puedes usar la versión para tu SO preferido. Si el dispositivo real nos da problemas, podemos usar el Emulador de Android o uno de terceros.

NOTA: Necesitamos conexión a Internet durante la instalación, ya que, según las opciones, descargará componentes para completar el proceso.

En el ejemplo se ve la instalación de una versión anterior, aunque la nueva se instala prácticamente igual. Puedes ver una guía en este enlace.

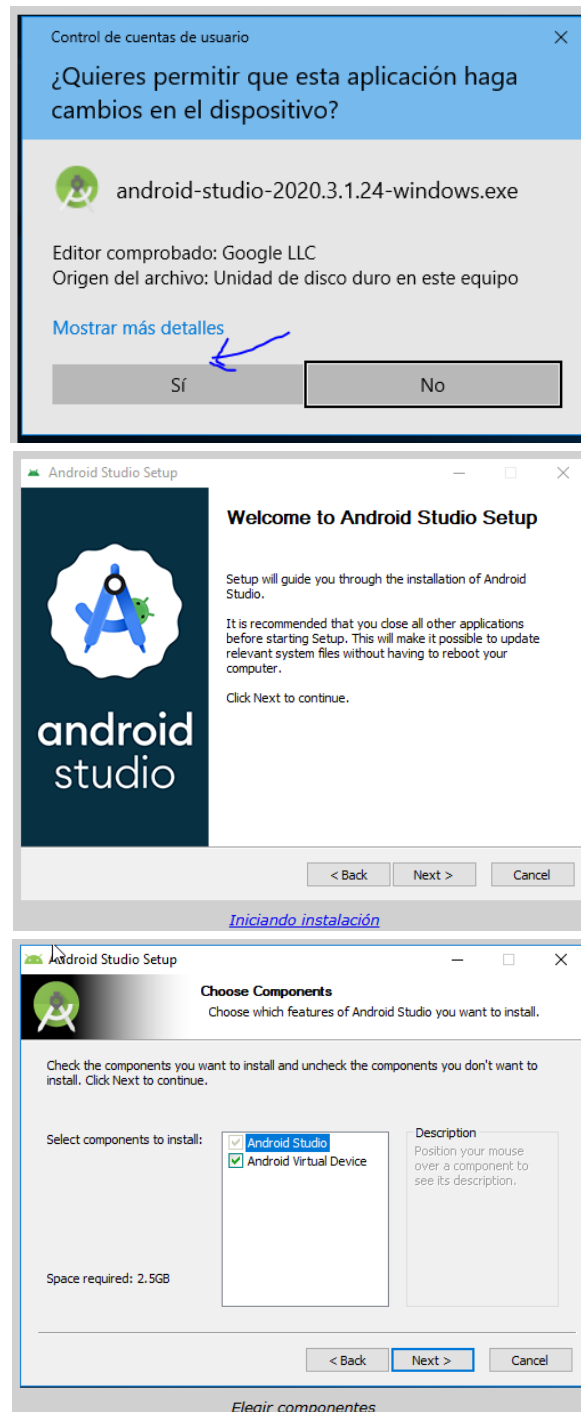
Una vez descargado android-studio-2024.2.1.11-windows.exe, clic derecho y "Ejecutar como administrador"; con doble clic daría error, como se muestra abajo.

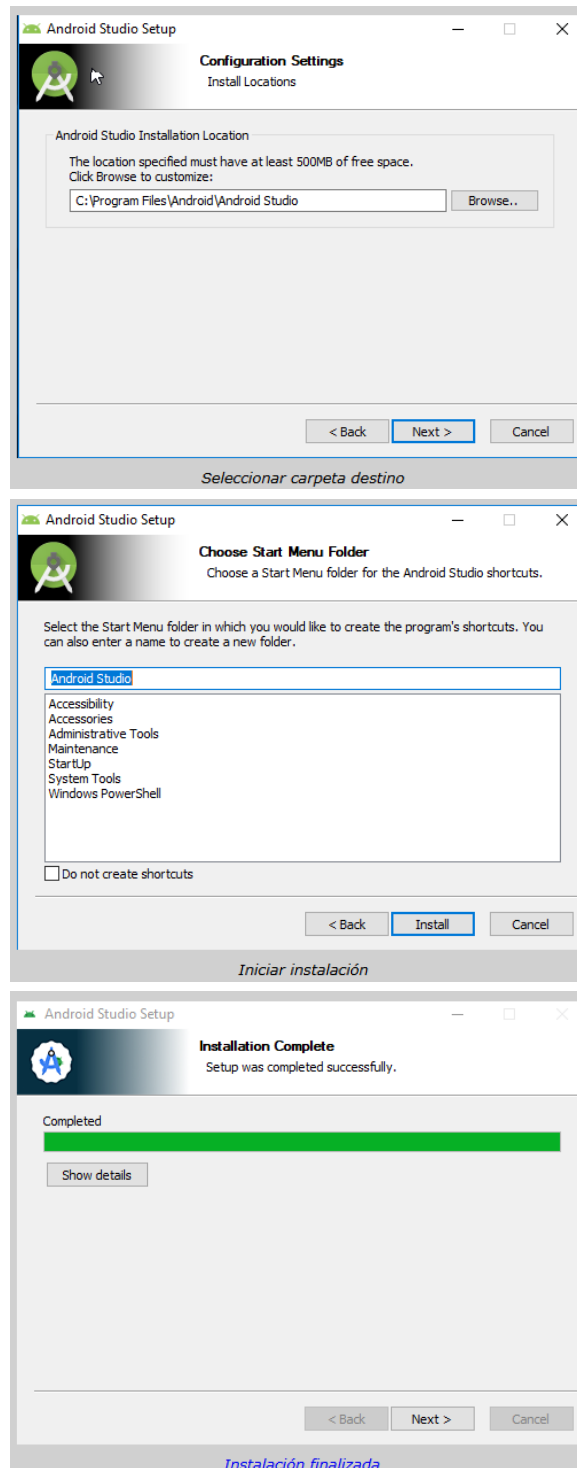


¡Atención! Esta parte puede variar en la versión más nueva; si tienes algún problema al instalar, dímelo.

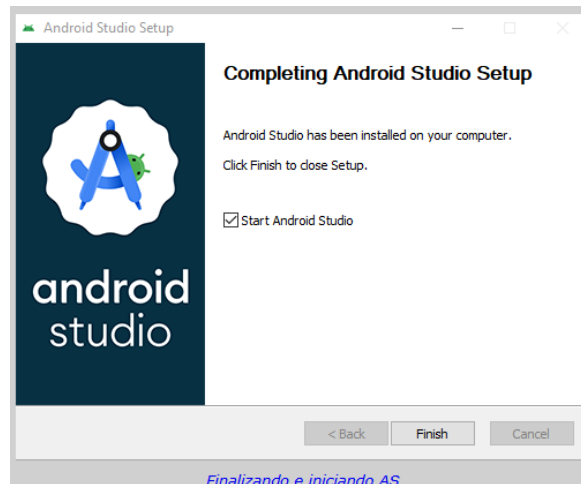
Selecciona SÍ para permitir cambios, luego marca ambos componentes (el propio AS y el AVD, el emulador de Android). Pulsa Siguiente, elige la carpeta de destino

y pulsa de nuevo Siguiente, luego Instalar para comenzar



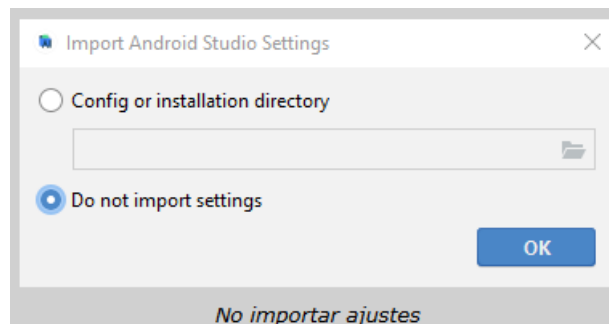


Al terminar, pulsa Siguiente y luego Finalizar para iniciar AS.

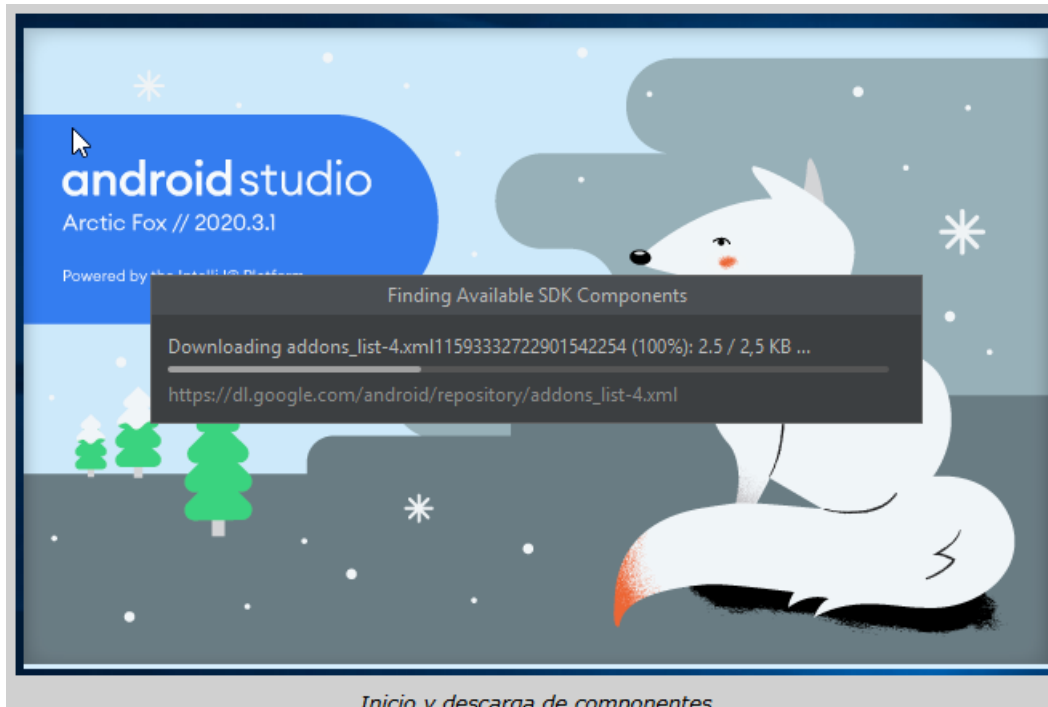


Finalizando e iniciando AS

La primera vez que inicia AS, pregunta si queremos importar ajustes de otra configuración. Deja "Do not import settings".

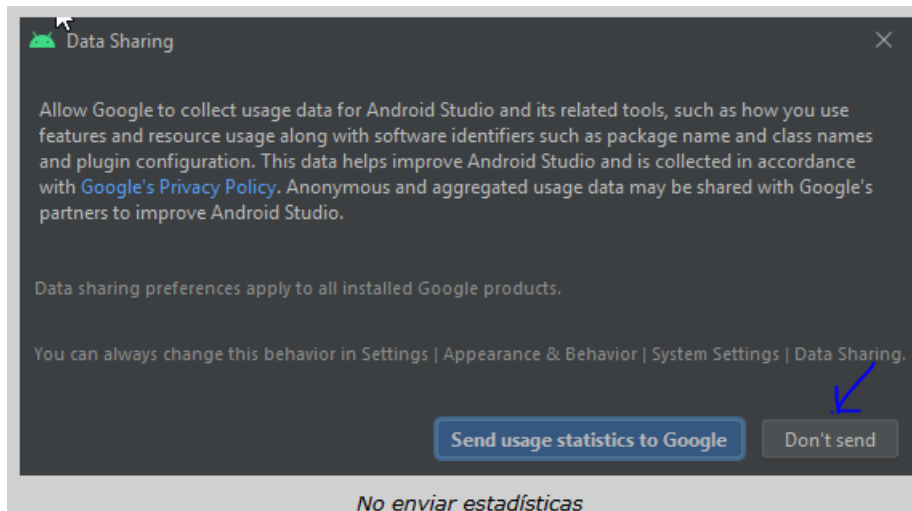


No importar ajustes

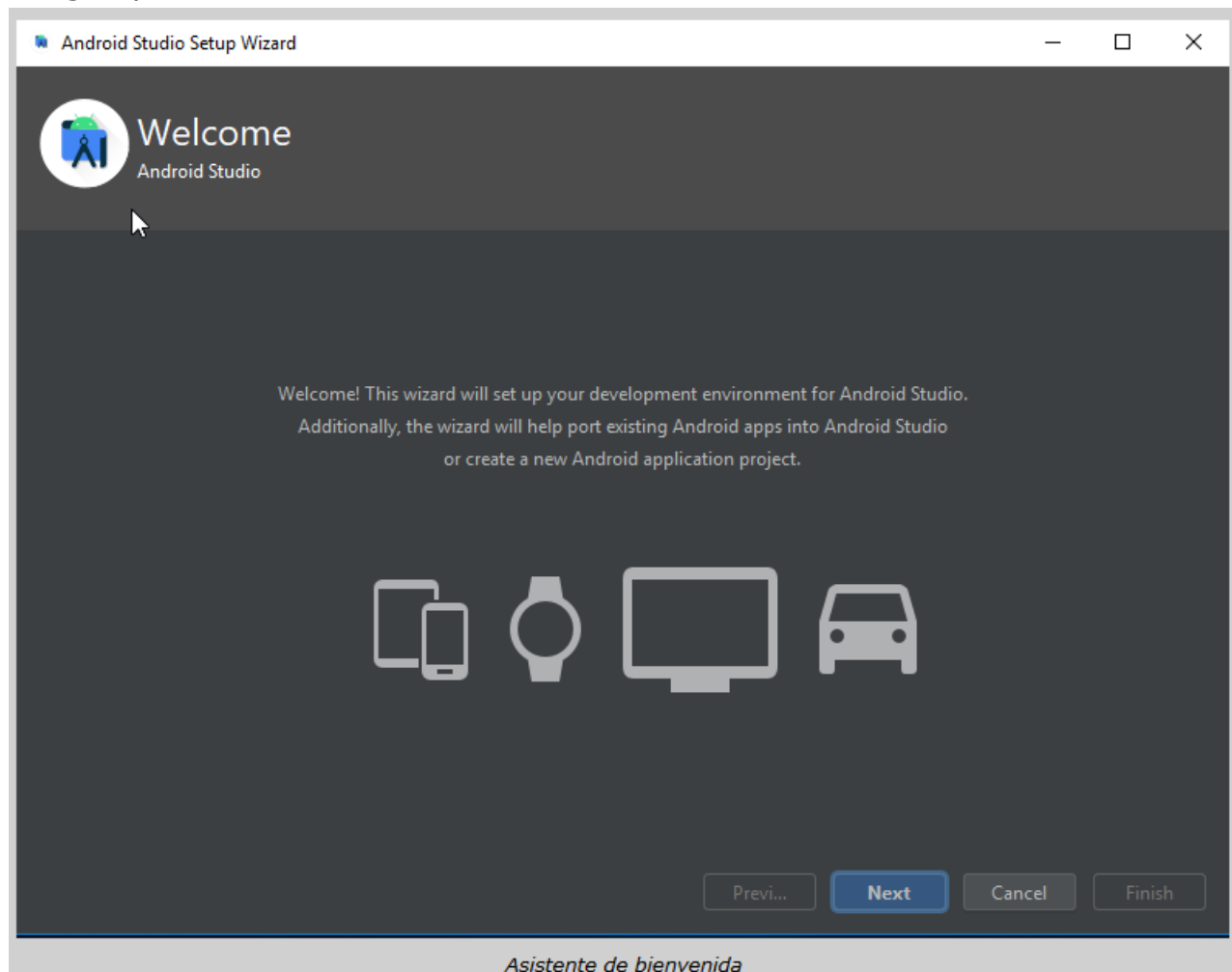


Inicio y descarga de componentes

Preguntará si queremos compartir datos: mejor no. Pulsa Don't send.

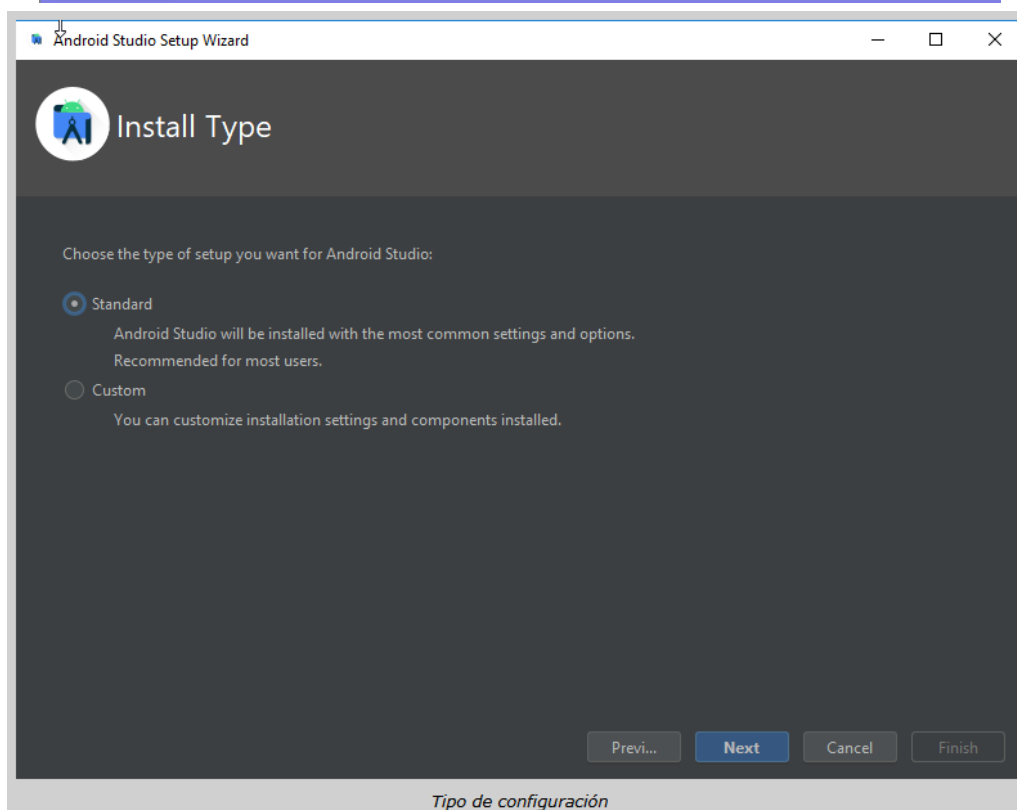


Luego aparece el Asistente de bienvenida.

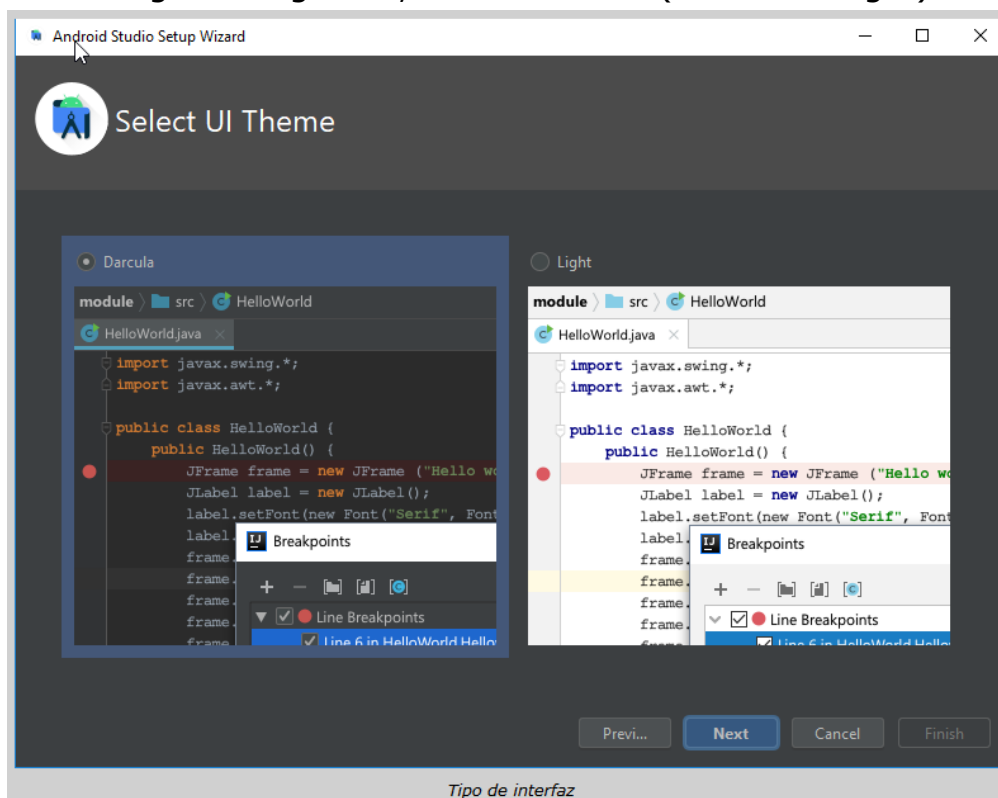


Pulsa Siguiente; preguntará si queremos instalación estándar o personalizada. Elegimos personalizada para asegurar que no instala nada innecesario y pulsamos Siguiente.

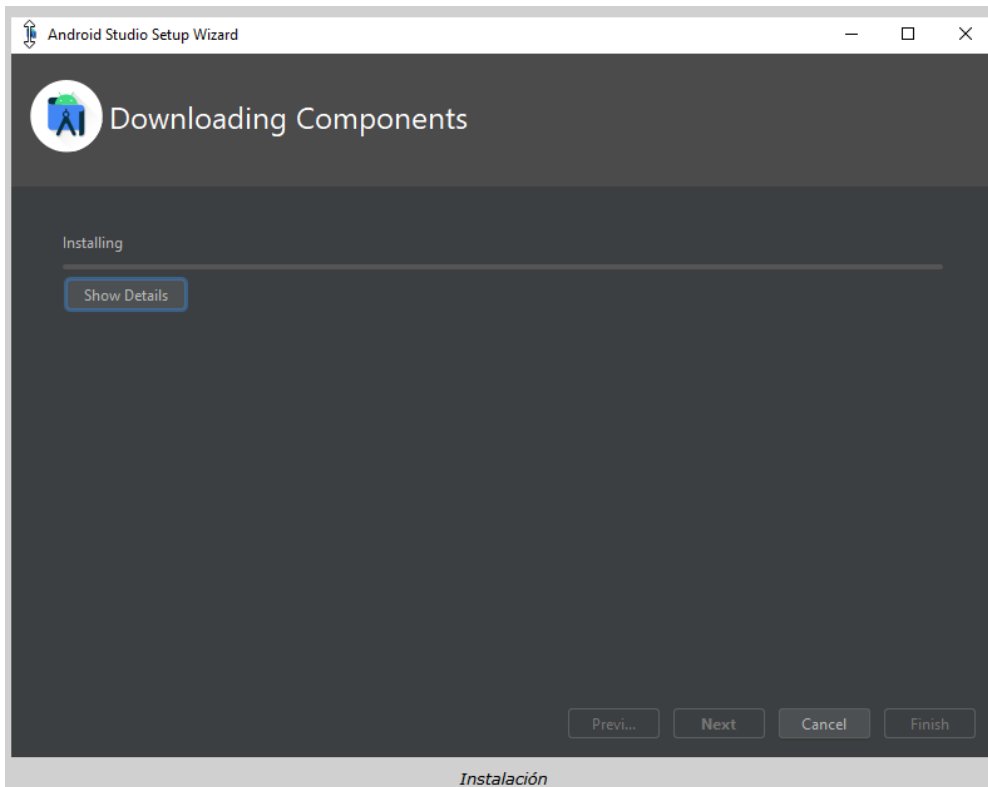
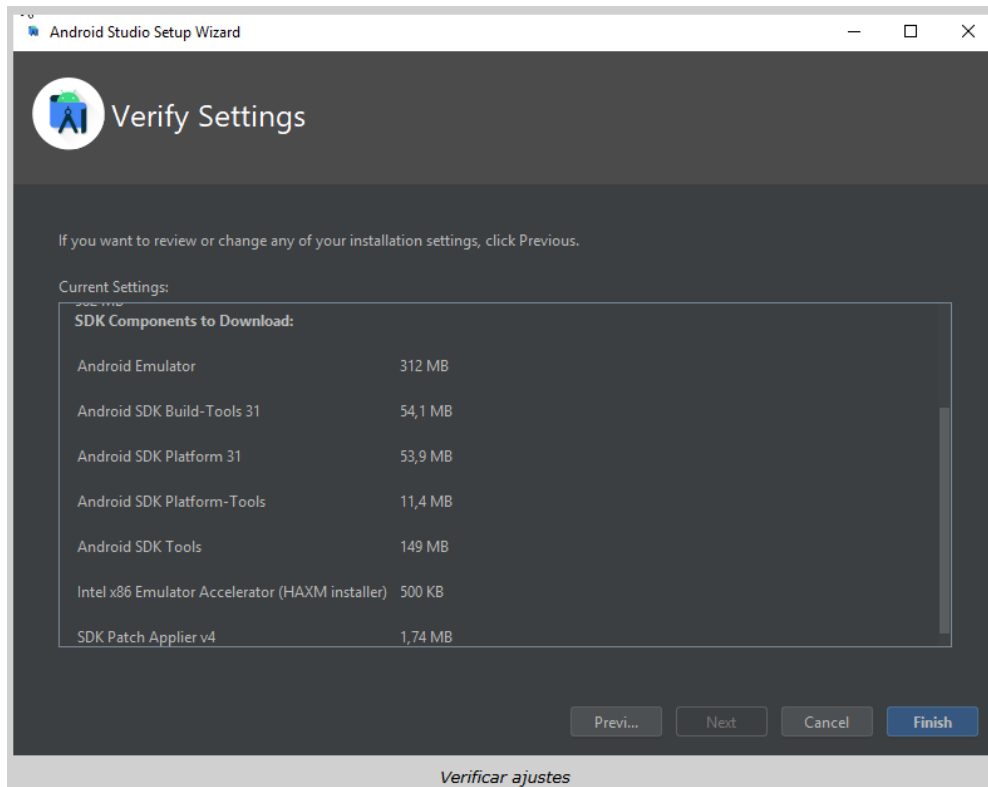
Si hay una actualización de plugins, actualízala primero.

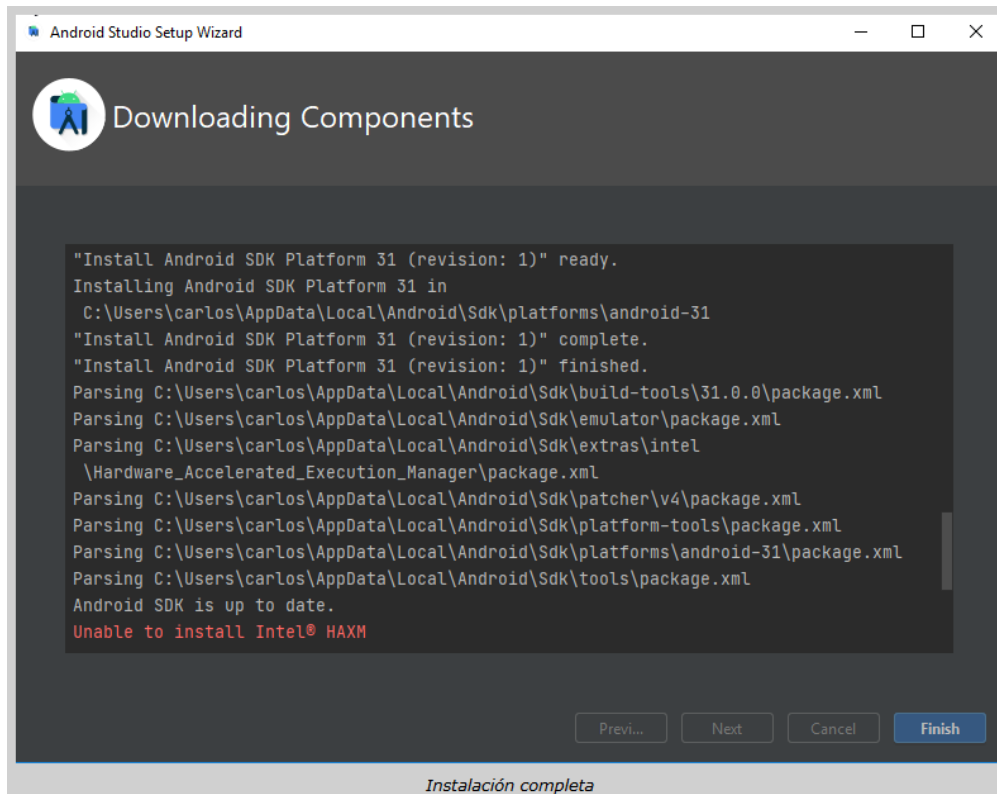


Luego permite elegir la UI gráfica, oscura o clara (Darcula o Light).



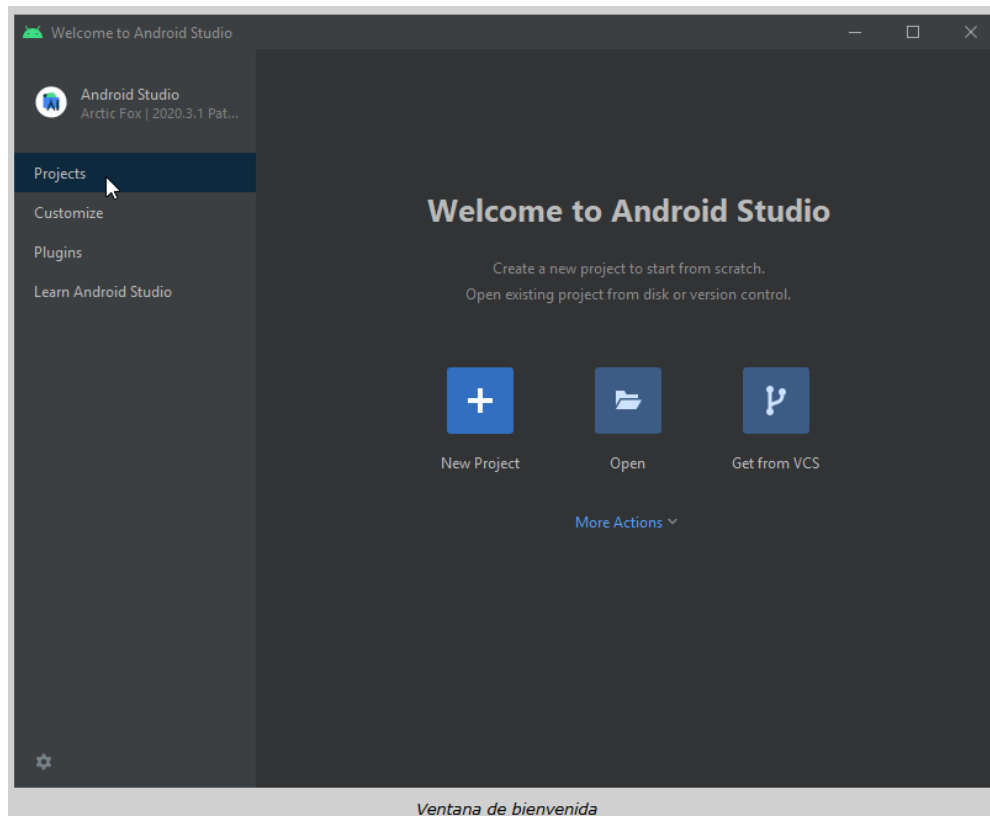
Verifica los ajustes y pulsa Finalizar.





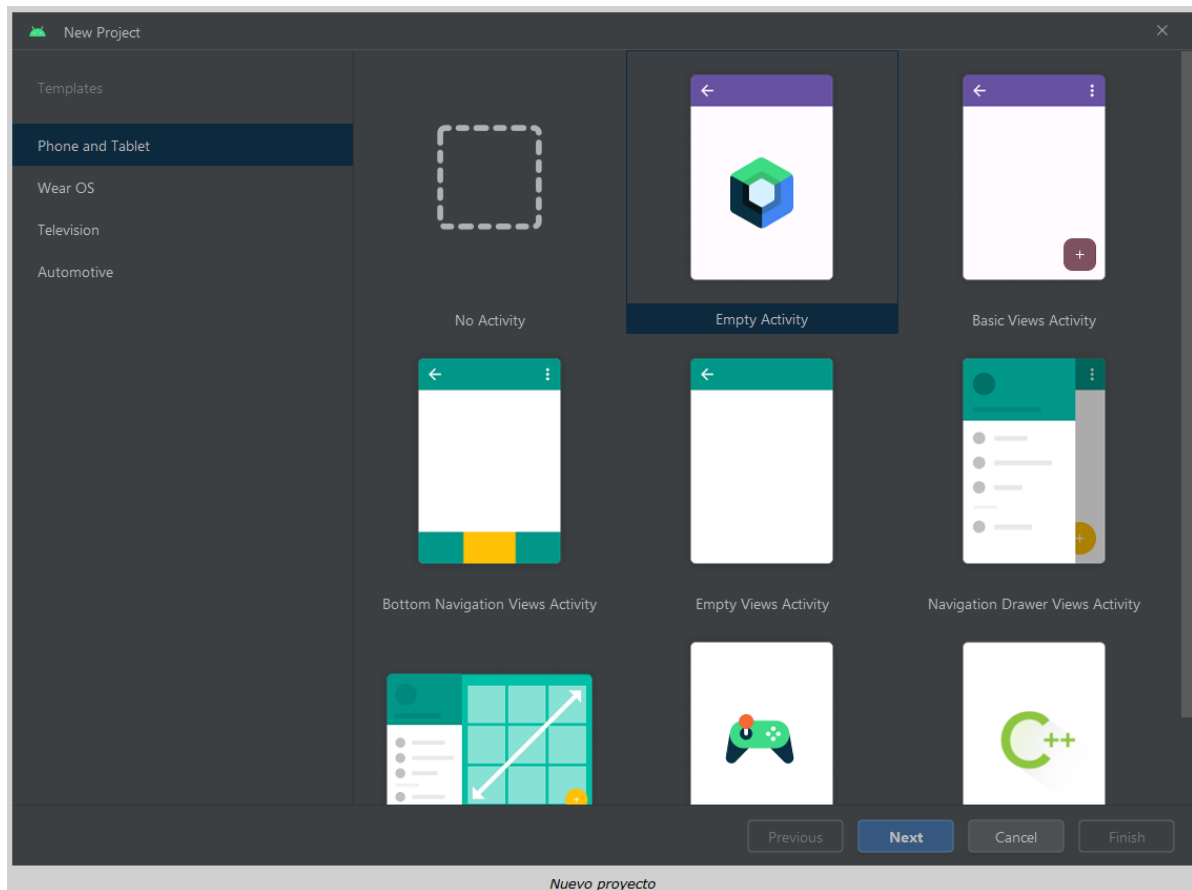
6.4 Hola Mundo. Mi primera aplicación

Ya con Android Studio instalado, creemos nuestra primera app "Hola Mundo" para comprobar que todo funciona.

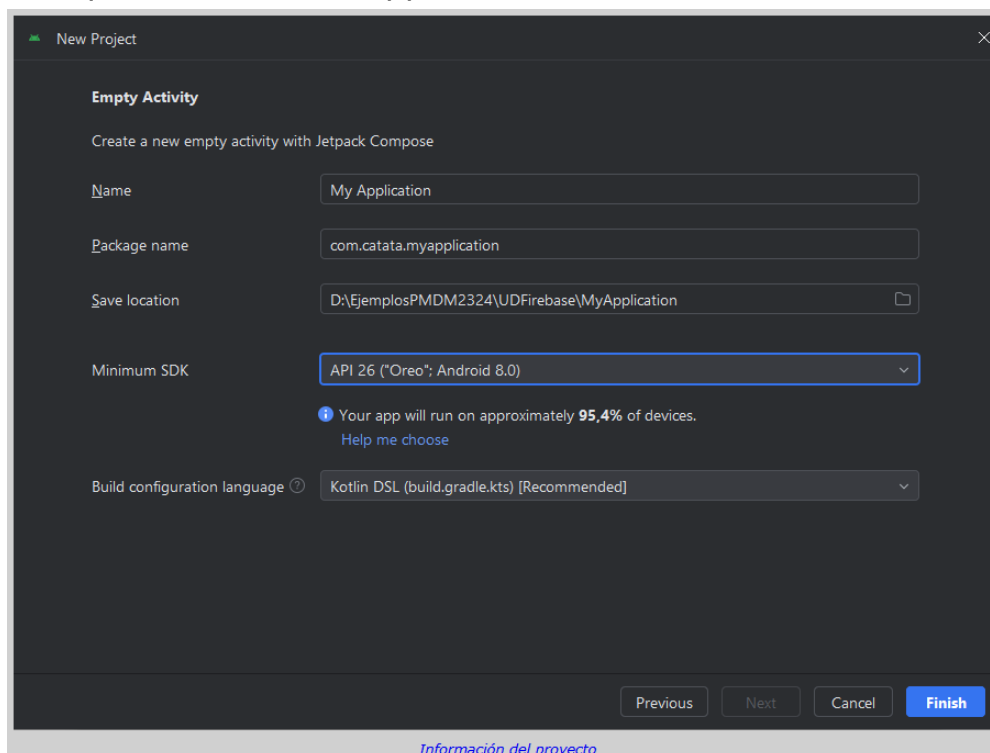


Inicialmente, elegimos New Project.

El asistente permite crear proyectos para varios tipos de dispositivos: móviles, tablets, wearables, Android TV, automoción... Pero en este curso solo crearemos apps para móviles o tablets, así que mantenemos Phone and Tablet (lado izquierdo). A la derecha hay plantillas básicas según el tipo de app. Para aprender, normalmente elegimos Empty Activity.



En la siguiente pantalla configuramos nombre del proyecto, ubicación de guardado, nombre de paquete, lenguaje (Java o Kotlin —por defecto—) y el SDK mínimo en el que funcionará la app.



Nombre de la App: Hello World

Nombre de paquete: com.xxyyzz.projectname, donde xx son las dos primeras letras de tu nombre, yy las dos primeras de tu primer apellido, y zz las dos primeras de tu segundo apellido. Por ejemplo, si mi nombre es Rosa García Pérez, entonces xxyyzz = rogape.

Usaremos com.xxyyzz para todos nuestros proyectos como nombre de paquete; no olvides configurar el nombre del proyecto.

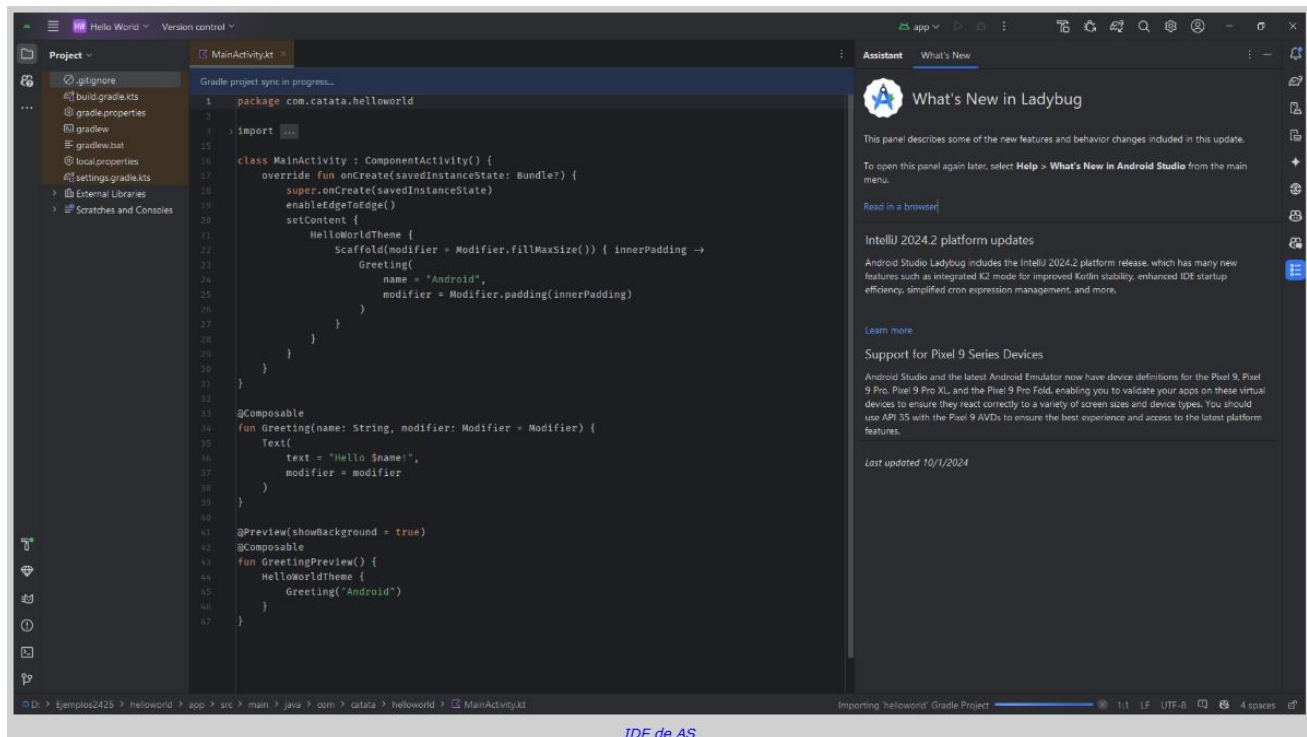
Carpeta de destino: La carpeta donde se guardará el proyecto (puedes dejar la predeterminada).

Lenguaje: Java o Kotlin. Seleccionamos Kotlin.

SDK mínimo: De momento, el valor por defecto: API 26: Android 8.0 (Oreo), que funciona en ~95,4% de dispositivos. Como ves, cuanto más nuevo sea el SDK mínimo, menos dispositivos podrán usarlo.

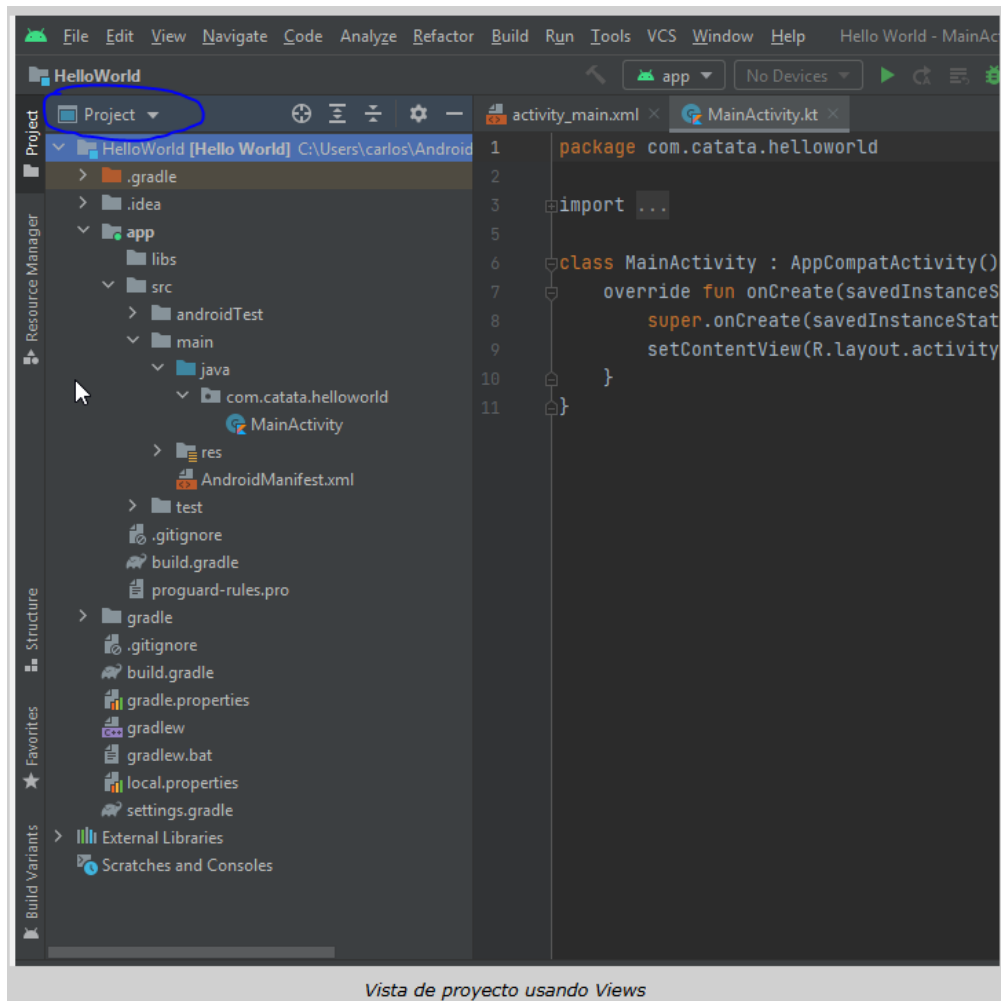
Si seleccionas el SDK más reciente, tendrás más funciones disponibles, pero solo funcionará en ciertos dispositivos.

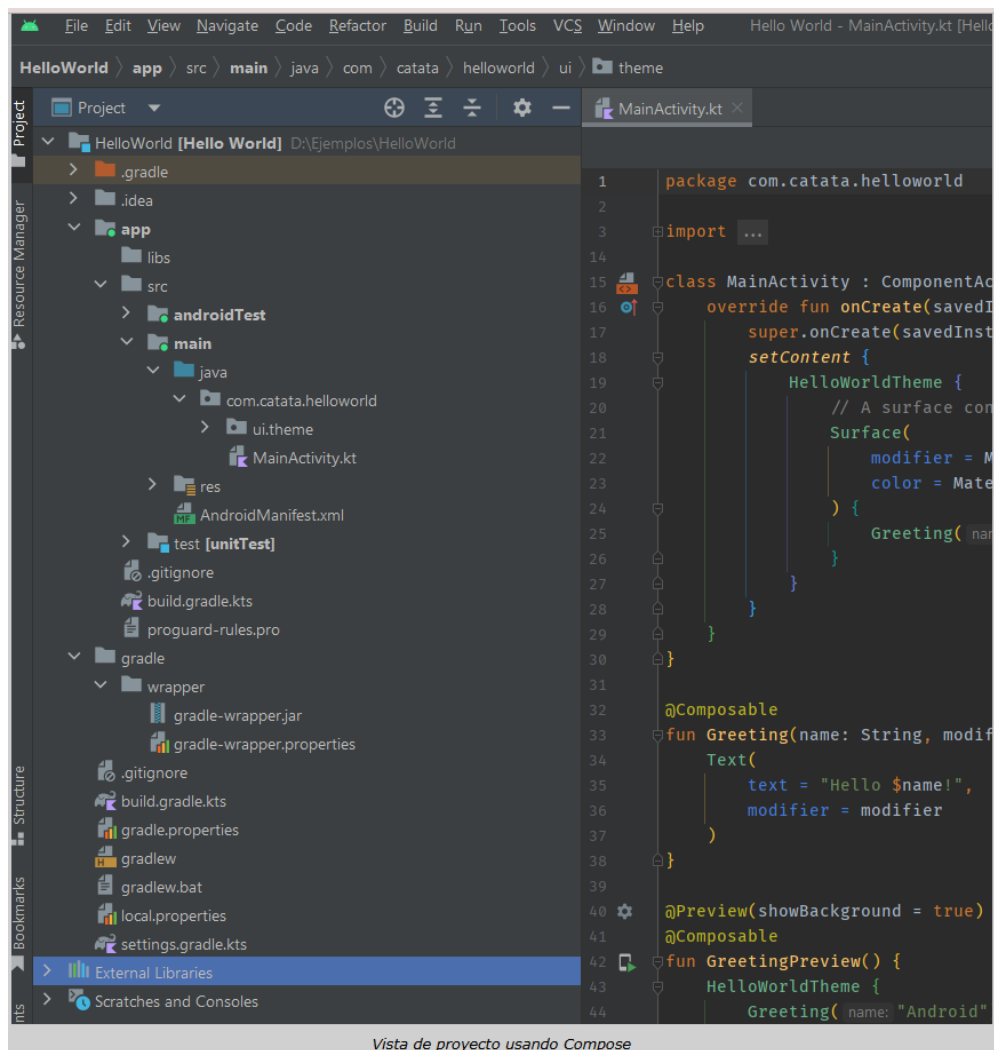
Tras unos “segundos”, aparecerá el IDE. Dedicamos unos minutos a explorar menús.



Hablemos de los archivos y carpetas principales del proyecto. Selecciona la vista de proyecto y expande todo.

Cada vez se usa más Jetpack Compose, por lo que las carpetas y archivos difieren un poco respecto a la estructura con vistas. En el sistema de vistas teníamos el concepto de layout, donde definíamos la apariencia en XML. Con Compose manejamos componentes como en otros lenguajes, por ejemplo Flutter o Swift UI.

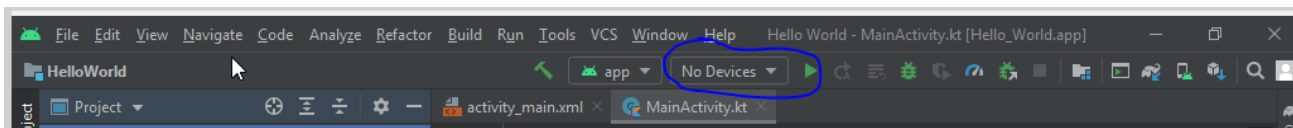




- Build: Subdirectorio que no debe modificarse; guarda los archivos compilados en formato dex, recursos preparados, etc. Se actualiza al modificar el proyecto.
- Libs: Librerías usadas por el proyecto.
- Src:
 - androidTest: Subdirectorio donde se construyen pruebas instrumentadas.
 - main:
 - java: Código fuente de la aplicación.
 - MainActivity.kt: Archivo Kotlin con el comportamiento de la pantalla principal.
 - ui.theme: Directorio con archivos relacionados con la apariencia (temas, colores, tipografías...). Por defecto: Color.kt, Theme.kt y Type.kt.
 - res: Recursos de la aplicación.
- AndroidManifest.xml: Archivo de configuración de la app.
- .gitignore: Archivo para repositorio Git si varias personas trabajan en el proyecto.
- build.gradle: Configuración de Gradle, la herramienta de compilación.

- proguard-rules.pro: Reglas de optimización/obfuscación.

Por ahora no podemos ejecutar la app porque no tenemos dispositivo. Podemos ejecutarla en un móvil real o en un emulador. Revisa la siguiente sección.



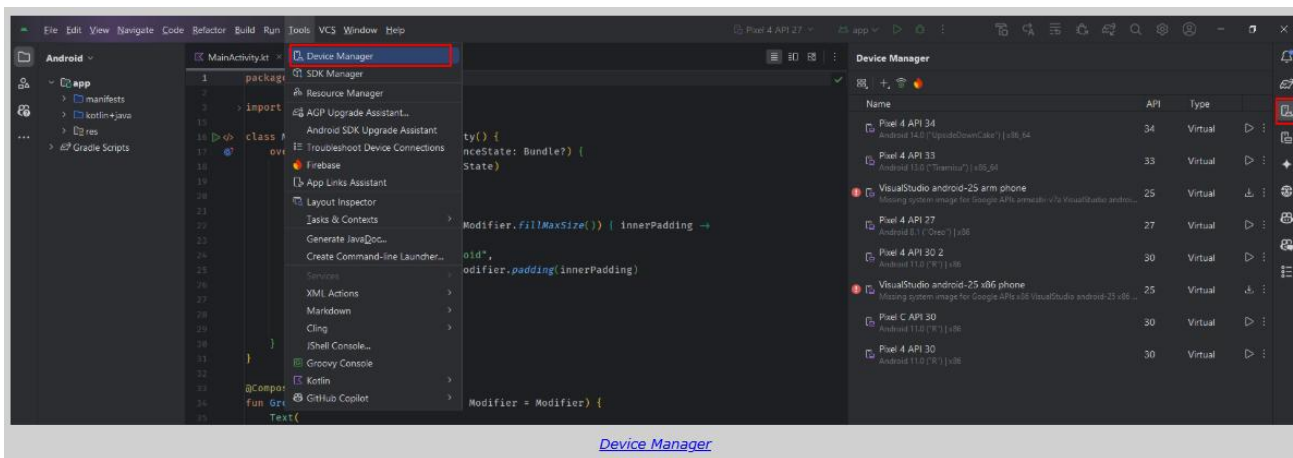
En versiones recientes suele venir un emulador por defecto. Si no, mira la siguiente sección.

7. EMULADOR DE ANDROID (AVD - ANDROID VIRTUAL DEVICE)

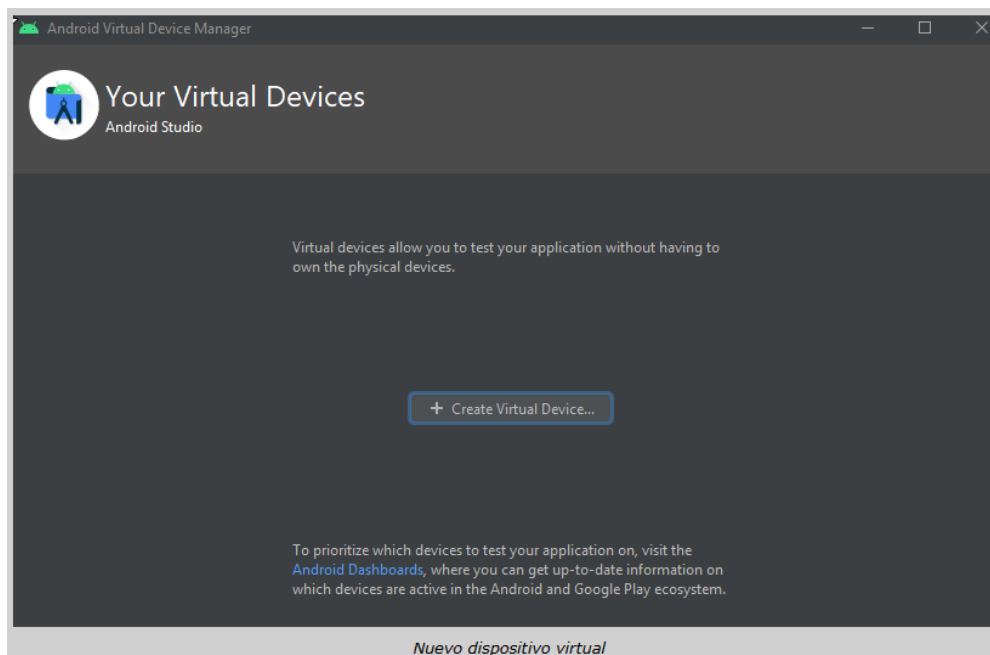
AS (Android Studio) incluye un emulador AVD integrado, que permite crear un dispositivo móvil virtual (teléfono, tablet, etc.) con características personalizables tanto de hardware (memoria, SD, etc.) como de software (versión de Android o nivel de API, etc.).

El AVD Manager permite crear dispositivos ajustados a nuestras preferencias y necesidades.

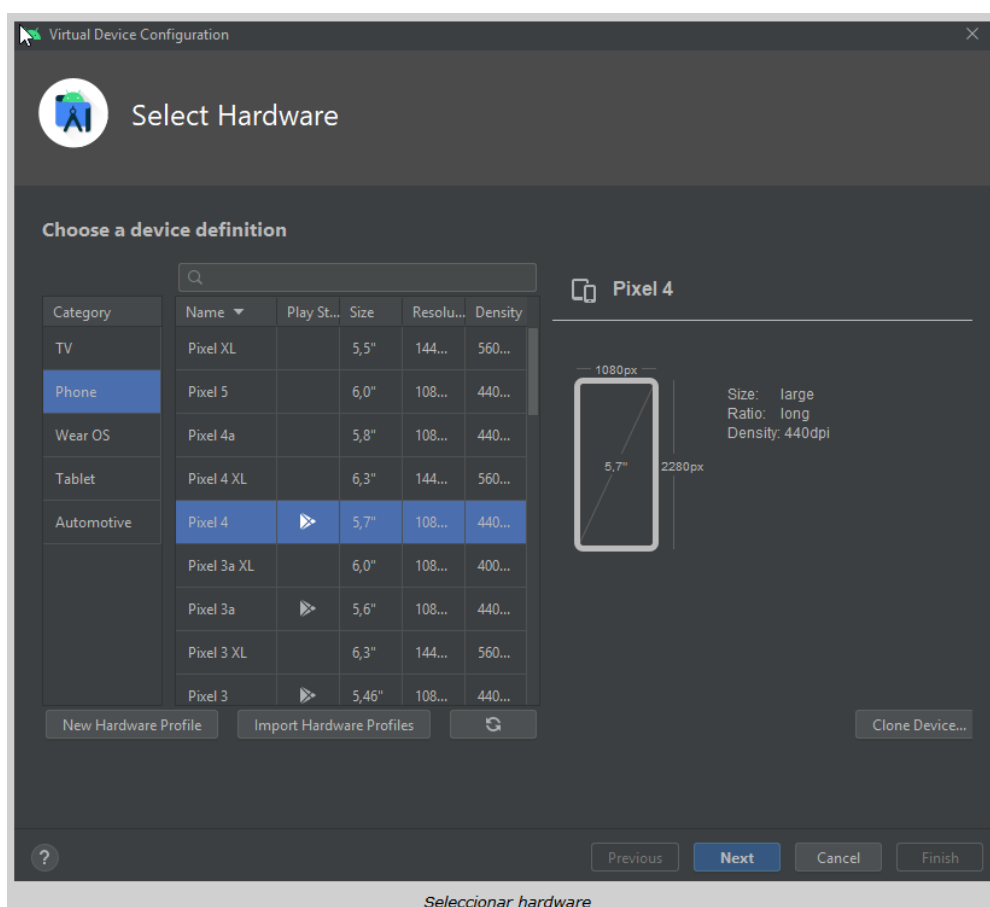
Abrir el AVD Manager.



Crear un nuevo dispositivo virtual



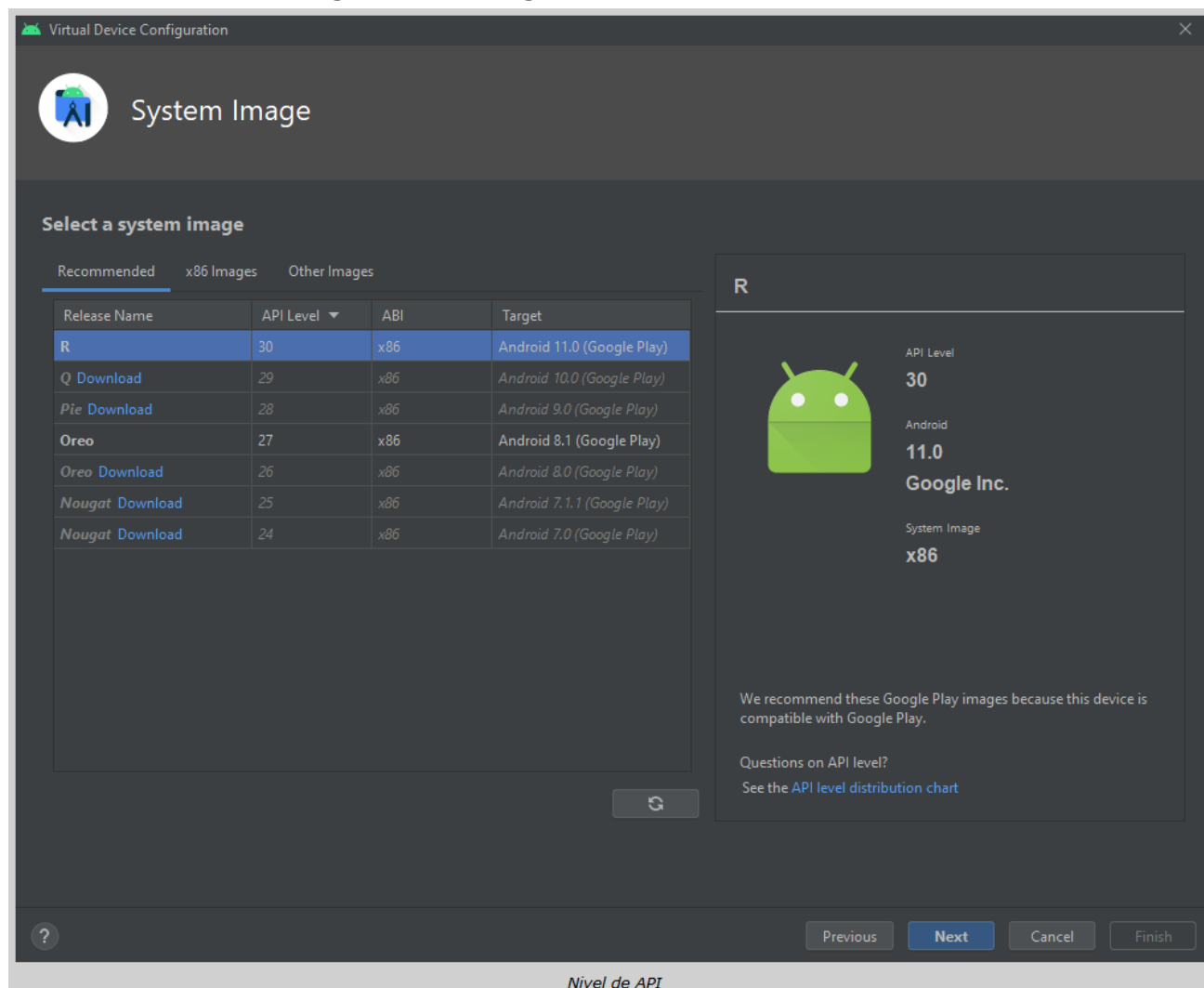
Podemos elegir un perfil existente o crear uno nuevo. Seleccionemos Pixel 4, que incluye Play Store y lo usaremos más adelante, así todos trabajamos con el mismo emulador.



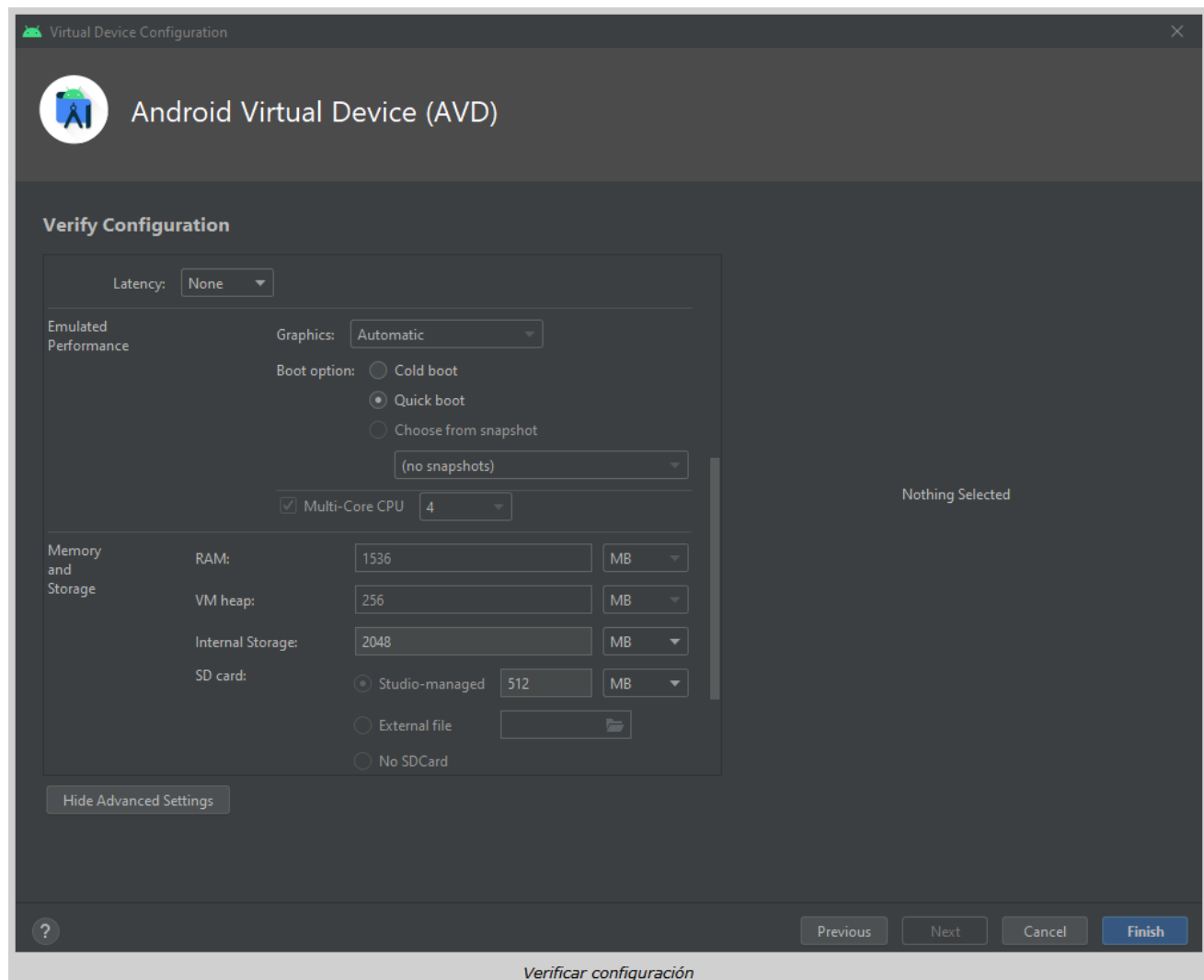
Debemos elegir el nivel de API; en este caso, elegiremos el más reciente, "Tiramisu" API 33, una de las versiones más actuales. Ten en cuenta que cuanto

más nueva sea la versión, más recursos consumirá y el emulador irá más lento.

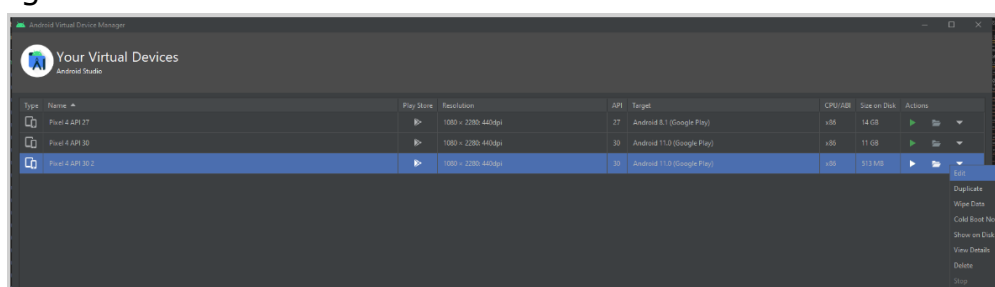
Si no la tienes descargada, descárgala antes.



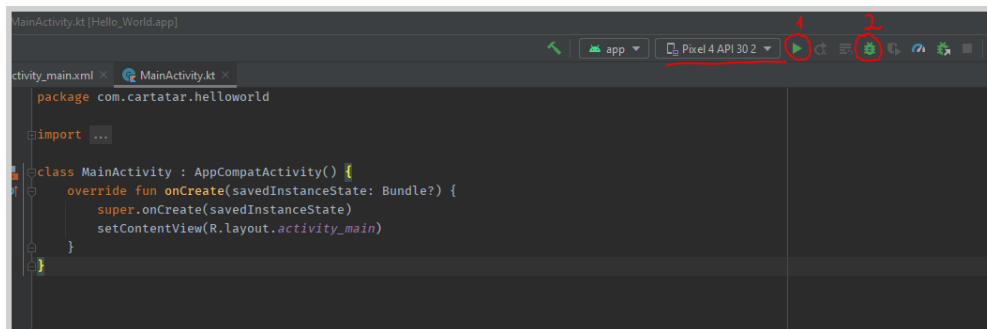
Pulsa Next para ir a la ventana de verificación de configuración, donde podemos cambiar el nombre, orientación inicial, emulación de cámara y red, almacenamiento interno, RAM... Pulsa Finish para crearlo.



El emulador se ha creado. Ahora puedes editarlo, eliminarlo o ejecutarlo desde el AVD Manager.

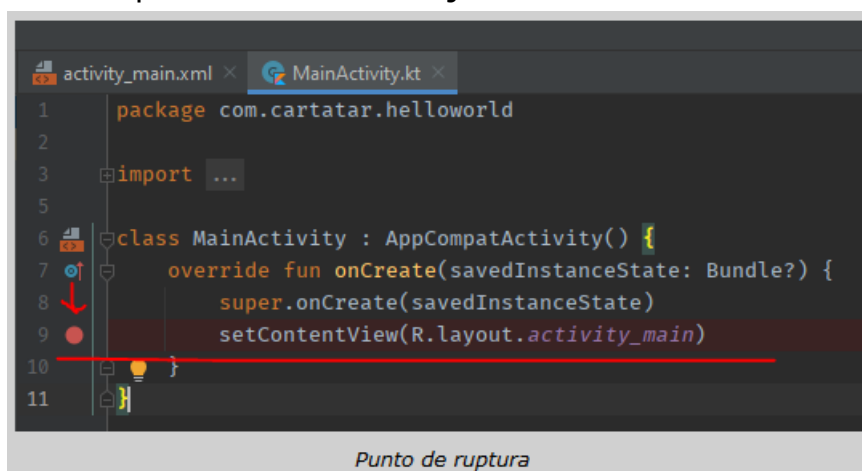


También podemos seleccionar este emulador para ejecutar nuestra app. Hagámoslo.



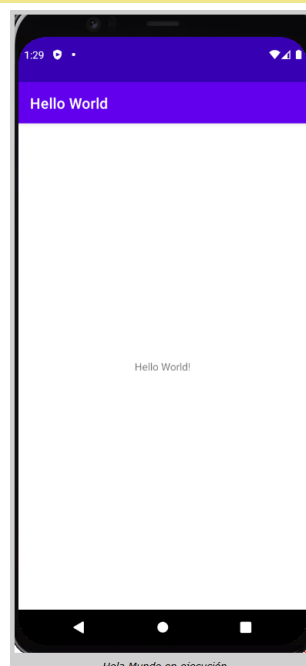
Pulsa Run App (1) para lanzar la app en el emulador seleccionado.

Pulsa Debug App (2) para depurar la app en el emulador seleccionado. Puedes poner un punto de ruptura haciendo clic junto al número de línea.



Si todo funciona, se abrirá el emulador y la app se ejecutará.

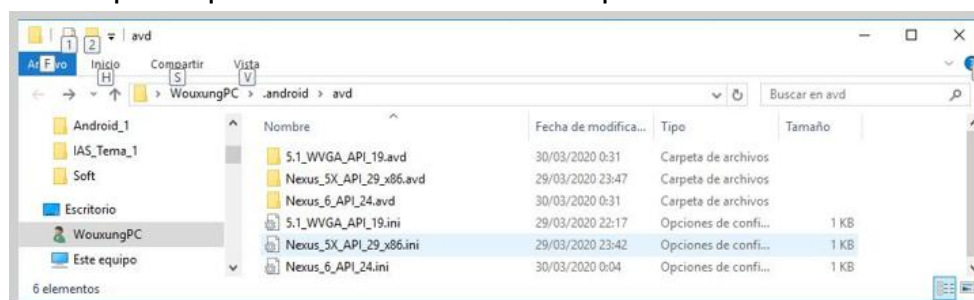
La primera vez que inicia el emulador, tarda un poco.



URL: Enlace oficial con información sobre AVD

<https://developer.android.com/studio/run/managing-avds?hl=es>

NOTA: Las máquinas virtuales de Android, como VirtualBox, son carpetas que podemos respaldar o compartir. Se guardan en la ruta del usuario seguida de *\.android\avd (en mi caso, C:\Users\javier\.android\avd). Para copiarlas, necesitamos la carpeta y su archivo *.ini correspondiente, como se ve abajo. Debes refrescar para que AS reconozca una copiada manualmente.

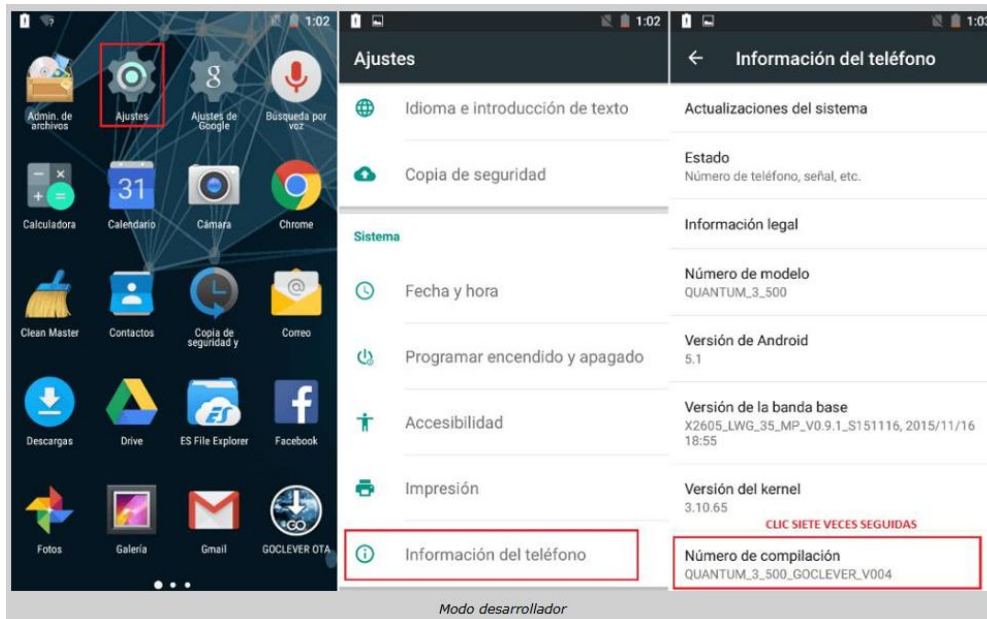


8. PRUEBAS EN DISPOSITIVOS REALES

8.1 Instalación de driver USB, Modo Desarrollador y Depuración USB

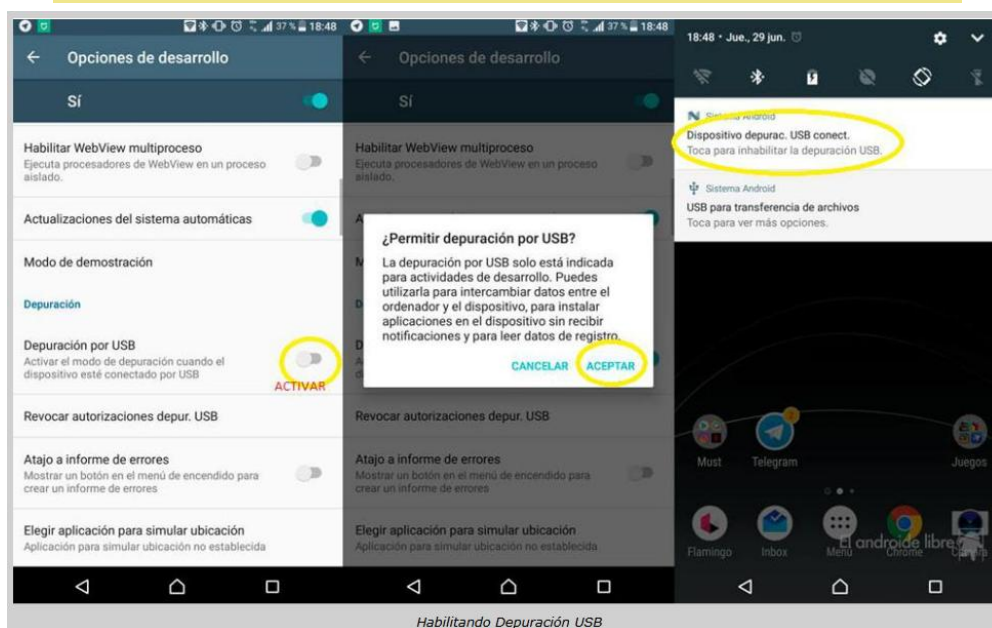
Aunque Android Studio permite probar nuestras apps con el emulador AVD, nada supera probarlas en nuestro dispositivo real (móvil, tablet, etc.), o incluso en varios con distintos tamaños y/o versiones de Android. Podemos usar drivers del fabricante, los de Google o los conocidos drivers ADB universales. Eso no impide que sigamos probando en diferentes dispositivos, pantallas, versiones, etc. El emulador AVD también se usa mucho.

Antes de instalar el driver, debemos poner el/los dispositivos Android en modo desarrollador. Es crucial, ya que permite probar y depurar directamente desde AS con el móvil conectado por USB. Para ello, ve a Ajustes, luego Acerca del teléfono, y toca Número de compilación siete veces. Verás un mensaje indicando que ya eres desarrollador.



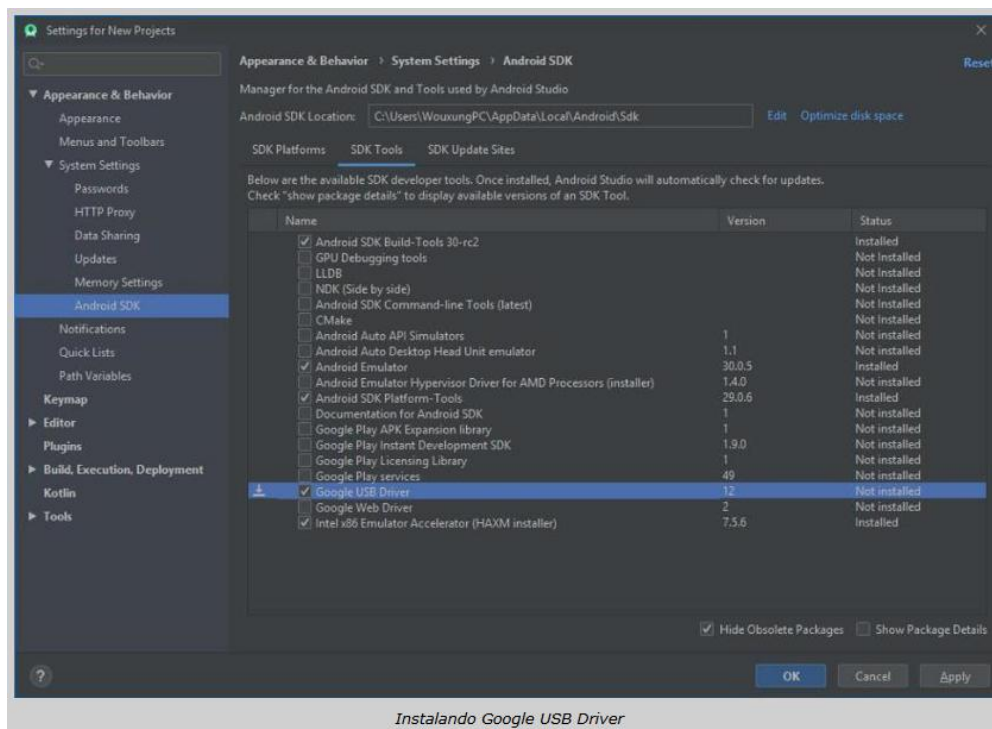
Volviendo a ajustes, debe aparecer un nuevo menú Opciones de desarrollador, donde debemos habilitar la Depuración USB.

Nota: En la imagen, la depuración USB está deshabilitada.



Ahora instalemos el driver USB del dispositivo. Si no lo tenemos, el fabricante puede proveerlo, o puede instalarse automáticamente mediante actualizaciones por Internet, o incluso pedirnos instalarlo al conectar el teléfono al PC.

Instalar antes los drivers de Google puede ayudar. Ve a Tools -> SDK Manager -> SDK Tools y marca Google USB Driver. Pulsa OK dos veces y Finish.



En mi caso conecté tres dispositivos distintos y funcionaron sin intervención, como se ve en el Administrador de dispositivos. En dispositivos modernos, Windows suele reconocerlos sin problemas.

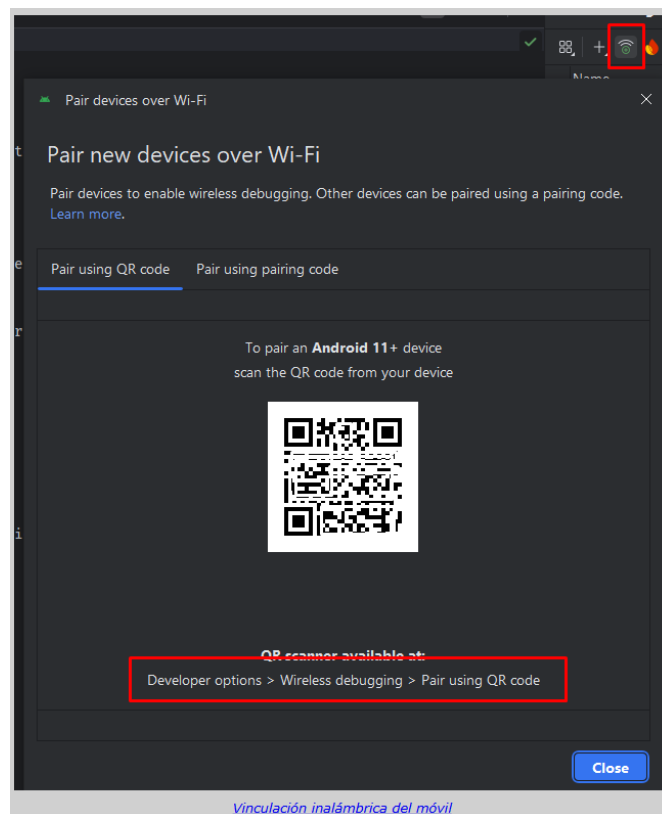


Otra opción es probar los enlaces oficiales de drivers OEM. Abajo algunos ejemplos.

OEM	URL del driver
Acer	http://www.acer.com/worldwide/support/
ALCATEL ONE TOUCH	http://www.alcatelonetouch.com/global-en/support/
Asus	https://www.asus.com/support/Download-Center/
BlackBerry	https://swdownloads.blackberry.com/Downloads/entry.do?code=4EE0932F46276313B51570F46266A608
Dell	http://support.dell.com/support/downloads/index.aspx?c=us&cs=19&l=en&s=dhs&~ck=anavml
Fujitsu	http://www.fmworld.net/product/phone/sp/android/develop/
HTC	http://www.htc.com/support
HUAWEI	http://consumer.huawei.com/en/support/index.htm
Intel	http://www.intel.com/software/android
Kyocera	http://www.kyocera-wireless.com/support/phone_drivers.htm
Lenovo	http://support.lenovo.com/us/en/GlobalProductSelector
LGE	http://www.lg.com/us/support/software-firmware
Motorola	https://motorola-global-portal.custhelp.com/app/answers/detail/a_id/88481/
MTK	http://online.mediatek.com/Public%20Documents/MTK_Android_USB_Driver.zip
Samsung	http://developer.samsung.com/galaxy/others/android-usb-driver-for-windows
Sharp	http://k-tai.sharp.co.jp/support/
Sony Mobile Communications	http://developer.sonymobile.com/downloads/drivers/
Toshiba	http://support.toshiba.com/sscontent?docId=4001814
Xiaomi	http://www.xiaomi.com/c/driver/index.html
ZTE	http://support.zte.com.cn/support/news/NewsDetail.aspx?newsId=1000442

8.2 Depuración inalámbrica

En versiones recientes de AS podemos depurar sin cable USB. Es muy simple: habilita la Depuración inalámbrica en modo desarrollador y escanea el código QR que te muestra AS.



Obviamente debes estar en la misma red que el ordenador y tu dispositivo real debe usar Android 11 o superior.

9. EMULADORES DE TERCEROS

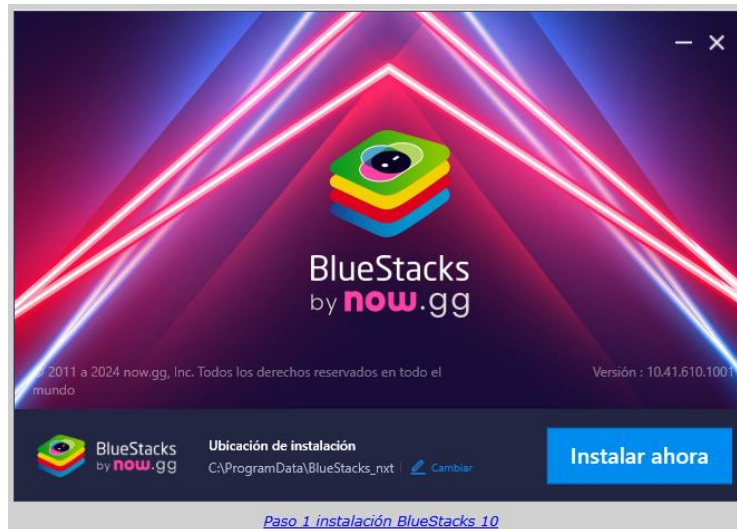
9.1 BlueStacks 10

Vamos a descargar BlueStacks desde:

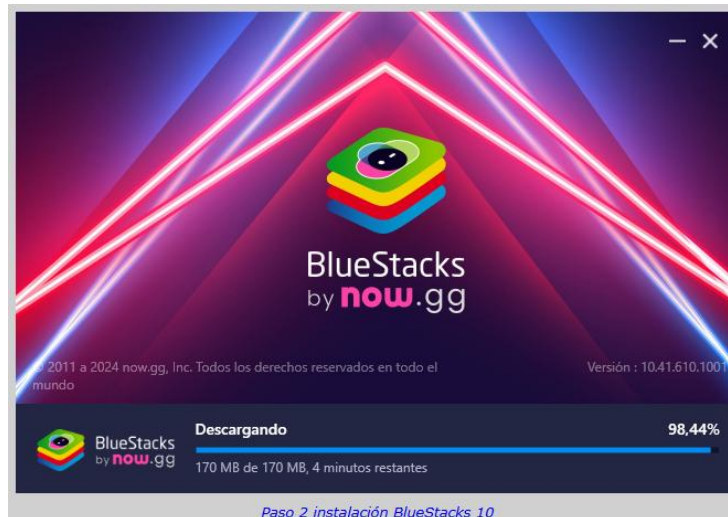
<https://www.bluestacks.com/es/index.html>

Una vez descargado el instalador, lo ejecutamos.

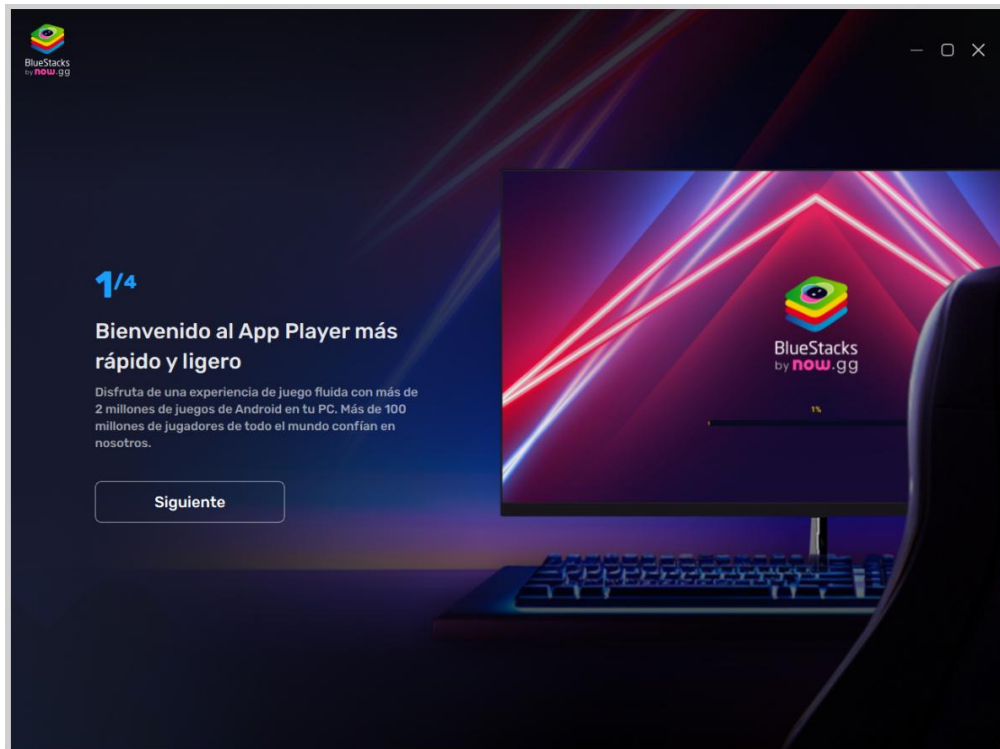
BlueStacksInstaller_10.190.0.5002_native_29f58463653ea660bf50c11c52fa2a25.exe es un instalador online. Permite que haga cambios.



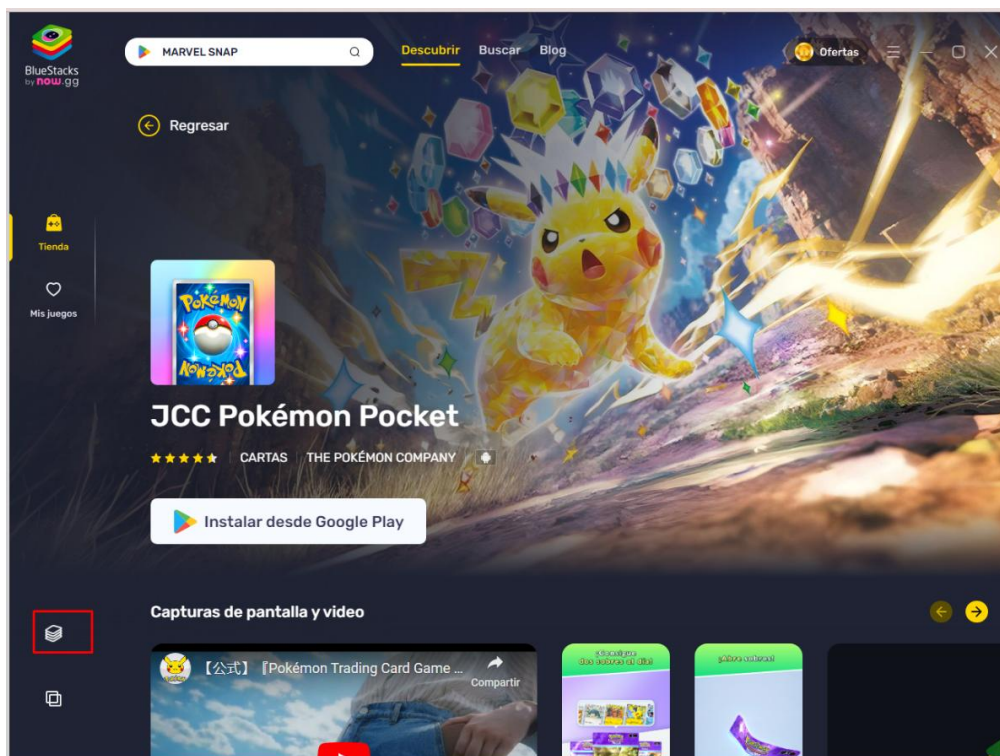
Haz clic en Install Now and accept terms

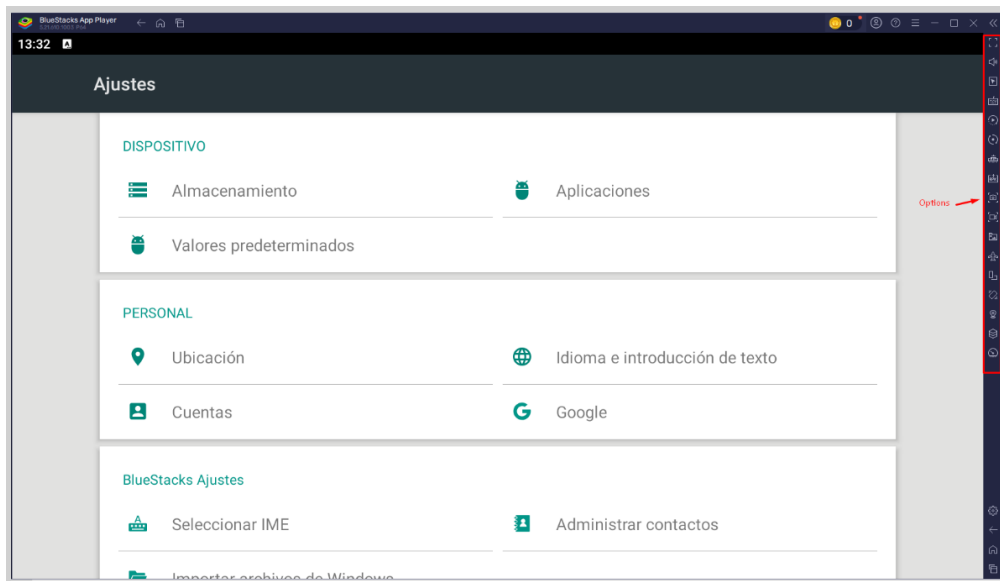


Completando el proceso de instalación. Pulsa Next



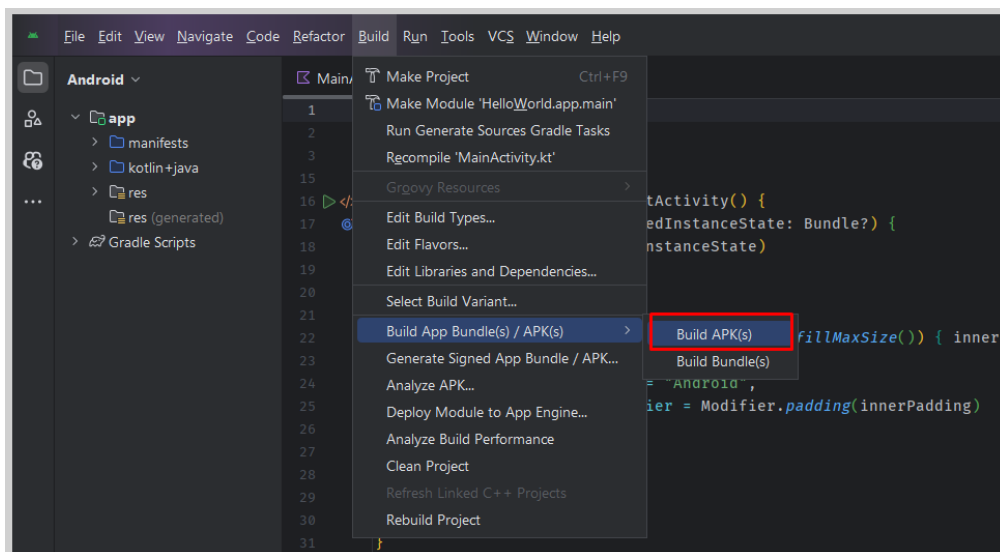
Ya podemos lanzar el emulador



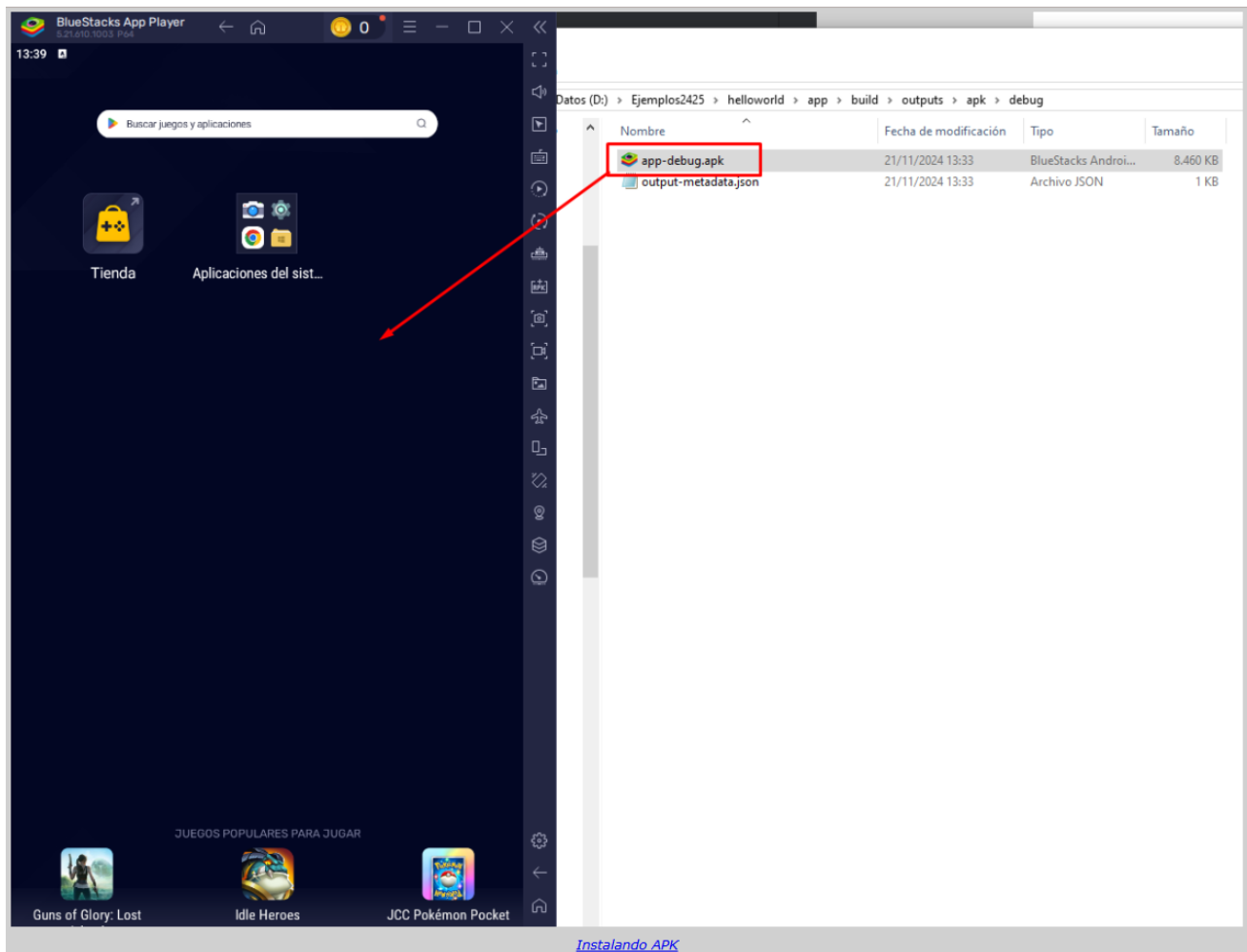
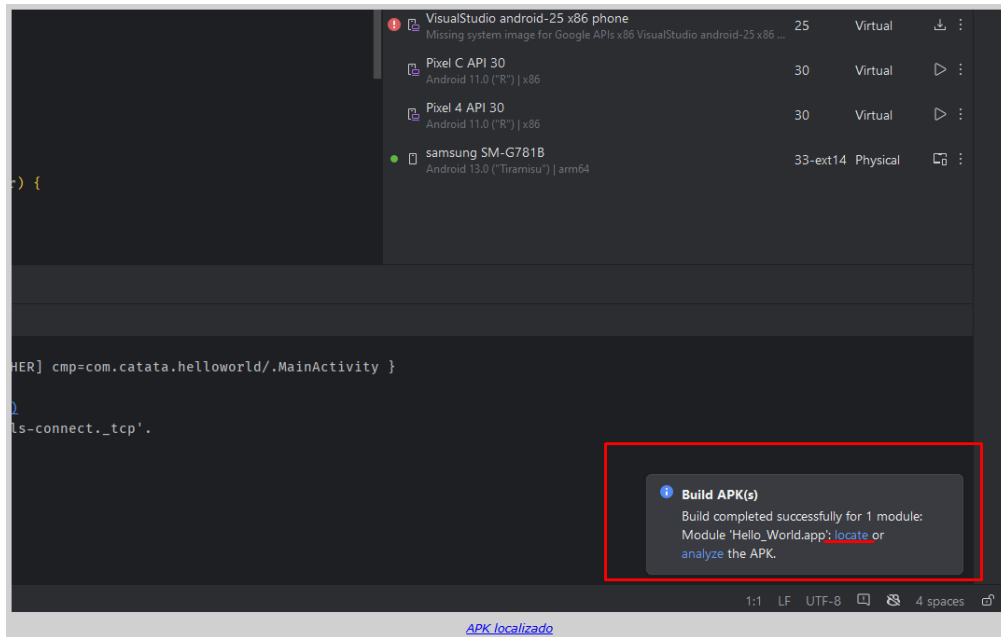


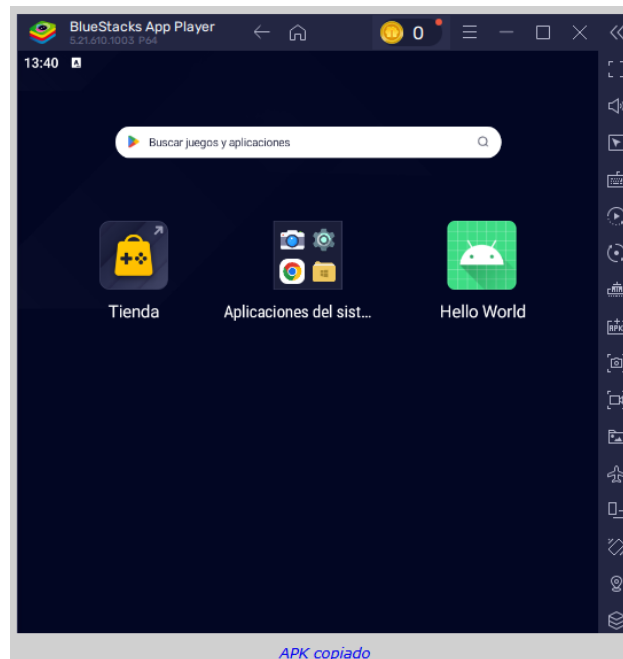
Una vez cargado, arrastra y suelta el APK directamente a la pantalla de inicio del emulador y se instalará.

Podemos obtener el APK desde AS en el menú Build

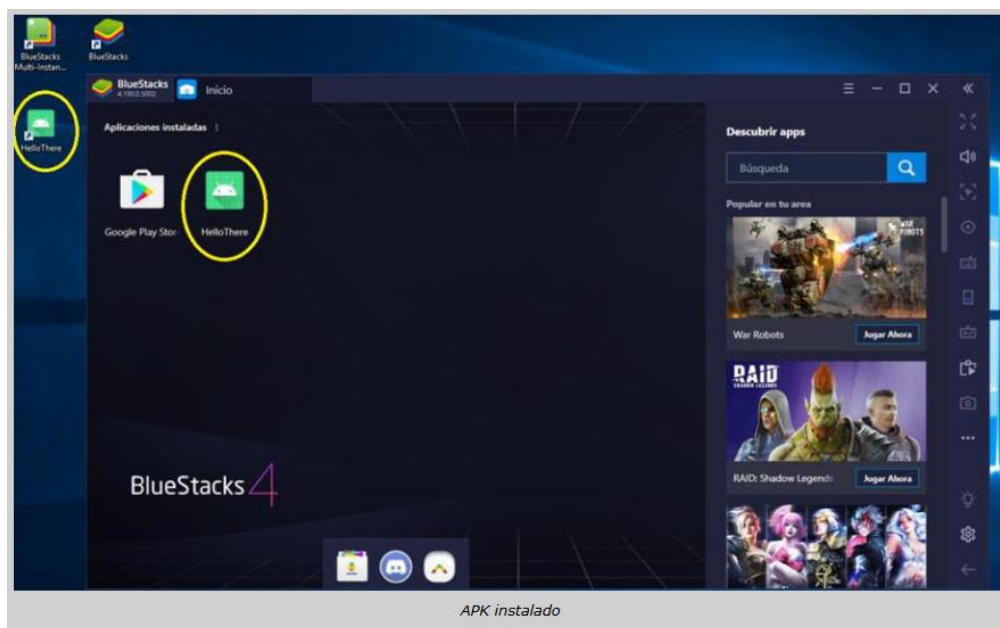


Tras compilar y generar el APK, AS muestra un mensaje con la ruta. Normalmente será Carpeta_Proyecto ⇒ app ⇒ outputs ⇒ apk ⇒ debug

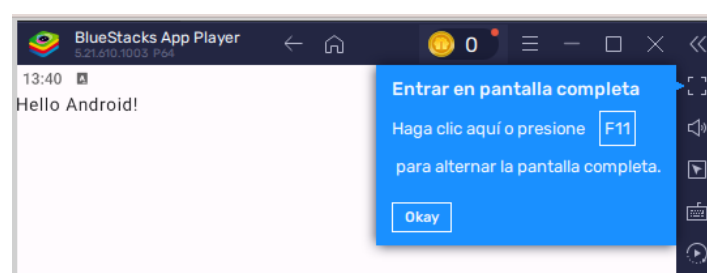




Además de instalarse dentro del emulador, también se creará un acceso directo en el escritorio.



Haz clic en el icono de tu app y... ¡Hola!



9.2 Genymotion

Sigue este enlace para instalar el emulador Genymotion e integrarlo con Android Studio

<https://desarrollador-android.com/desarrollo/herramientas/herramientas-de-ayuda/genymotion-emulador/>

(enlace algo desactualizado, pero aún útil).

10. OTROS SISTEMAS.

El otro gran ecosistema móvil es, por supuesto, iOS de Apple.

Desarrollar apps iOS suele requerir un Mac con XCODE instalado.

En cuanto a lenguajes, se incluyen C++ y el propio Swift de Apple.

11. BIBLIOGRAFÍA

[1] Apuntes de Carlos Tarazona Tárrega (2025).